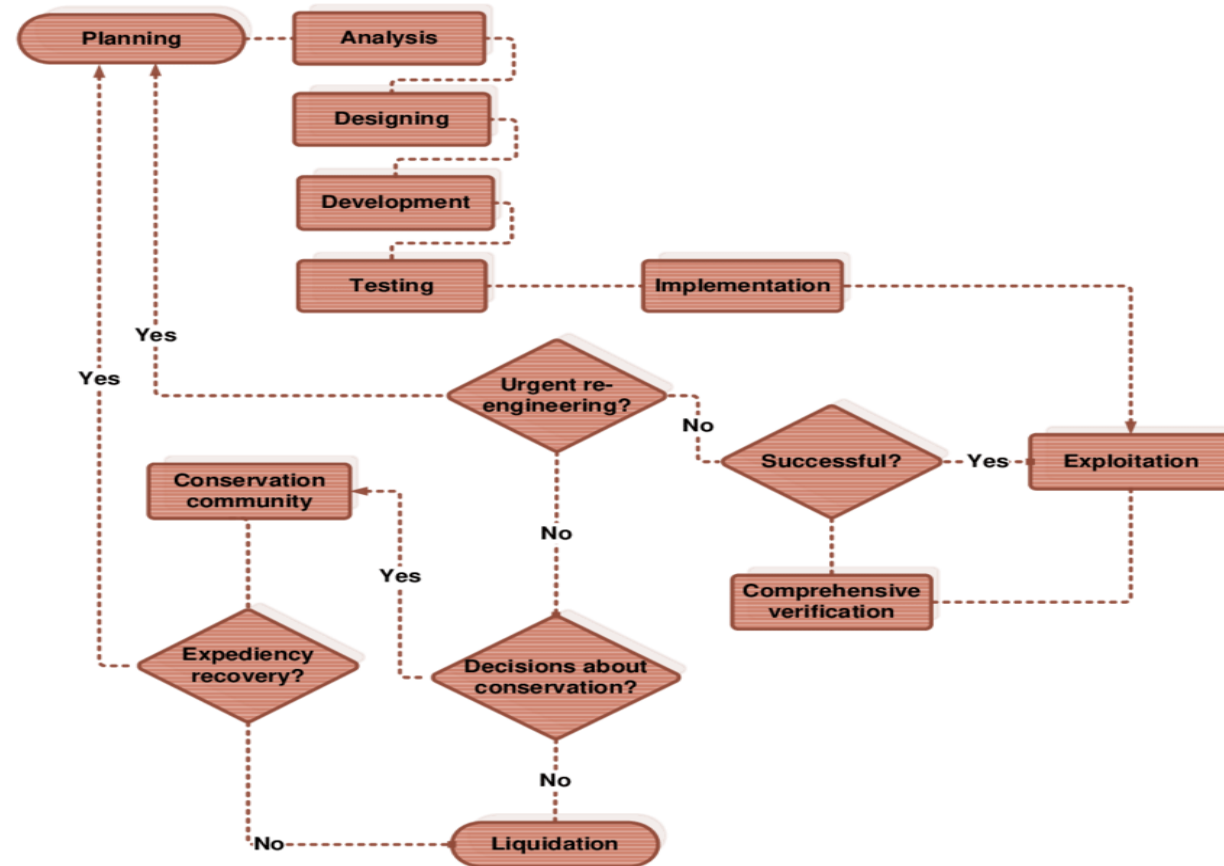


ALGORITMOS E LABORATÓRIO DE PROGRAMAÇÃO

Bases de Programação

- Professor: Fábio Contarini Carneiro
- E-mail: fabio.carneiro@uva.br

ALGORITMOS



Algoritmos

- Antes de programar com uma linguagem específica é necessário entender como trabalhar em um nível mais abstrato.
- Veremos o conceito de algoritmos e como usar algoritmos na programação de computadores.
- Uma definição genérica para algoritmo: “*Qualquer método utilizado para a solução de determinado problema*”.

Algoritmos

- A definição de algoritmo pode ser melhorada para: “*Uma sequência ordenada e sem ambiguidade de passos para a resolução de um problema*”.
- Sequência ordenada ... de passos
 - Cada passo deve contribuir para se chegar mais perto da solução final de um problema.
 - Cada passo deve avançar em direção à solução.
- Sem ambiguidades
 - Não deve haver dúvidas sobre o que cada passo significa para a resolução do problema.

Algoritmos

- A solução de um problema deve vir por passos.
 - Para problemas reais não existe atalho.
 - Não adianta tentar resolver um problema computacional indo direto à solução.
- Antes de programar deve-se pensar em uma solução na forma de algoritmo.
- Algumas propriedades:
 1. Cada operação deve ser bem definida.
 - Não deve haver dúvida sobre o seu significado. Não pode conter ambiguidades.
 2. Cada operação deve ser efetiva.
 - Deve contribuir para a solução do problema.
 3. Uma pessoa munida de papel e lápis deve poder seguir os passos do algoritmo.
 4. O algoritmo deve terminar em um tempo finito.

Algoritmos

- Exemplo: Algoritmo para fritar um ovo.
 1. Pegar um ovo.
 2. Colocar a frigideira no fogo.
 3. Colocar óleo na frigideira.
 4. Esperar o óleo esquentar.
 5. Quebrar o ovo separando a casca.
 6. Colocar o conteúdo do ovo na frigideira.
 7. Colocar sal.
 8. Esperar fritar.
 9. Passar o ovo da frigideira para um prato.

Algoritmos

- Exemplo: Algoritmo para procurar palavra em um dicionário.
 1. Abrir o dicionário.
 2. Ir para a primeira palavra.
 3. Ler a palavra.
 4. Se for a palavra procurada imprimir seu significado e terminar o algoritmo.
 5. Se não houver mais palavras informar que a palavra não existe e terminar o algoritmo.
 6. Ir para a próxima palavra.
 7. Ir para o passo 3.

Algoritmos

- No início da computação era comum comandos como “**ir para o passo 3**”.
- Com o aumento da complexidade dos programas, notou-se que estes comandos criavam um código difícil de corrigir.
- A solução encontrada foi usar blocos de comandos que são repetidos de acordo com uma condição.
- Permite melhor controle do fluxo de execução de um algoritmo.
 - Sempre se sabe qual parte do código está sendo executada.
- É conhecida como **Programação Estruturada**.

Algoritmos

- Exemplo: Algoritmo para procurar palavra em um dicionário.

Abrir o dicionário.

Ir para a primeira palavra.

Ler a palavra.

Repita:

 Se for a palavra procurada, faça:

 Imprimir seu significado.

 Terminar o algoritmo.

 Se for a última palavra, faça:

 Informar que a palavra não existe.

 Terminar o algoritmo.

 Ler a próxima palavra.

Algoritmos

- Exemplo: Algoritmo para calcular a soma dos N primeiros números.

Escrever “Entre com um número”

Ler N

Faça $A \leftarrow 1$

Faça Soma $\leftarrow 0$

Repita:

 Faça Soma $\leftarrow$ Soma + A

 Se $A = N$, faça:

 Escrever Soma

 Terminar o algoritmo

 Faça $A \leftarrow A + 1$

Algoritmos

- Exemplo: Algoritmo para calcular a soma dos N primeiros números (outra solução).

Escrever “Entre com um número”

Ler N

Faça $A \leftarrow 1$

Faça $Soma \leftarrow 0$

Enquanto $A \leq N$, faça:

 Faça $Soma \leftarrow Soma + A$

 Faça $A \leftarrow A + 1$

Escrever Soma

Algoritmos

- Exercício: Assumindo um computador que possua somente a operação aritmética de soma, fazer um algoritmo que realize a multiplicação de dois números inteiros positivos.

Ler A

Ler B

Faça $M \leftarrow 0$

Faça $C \leftarrow 1$

Enquanto $C \leq B$, faça:

 Faça $M \leftarrow M + A$

 Faça $C \leftarrow C + 1$

Escrever M

Algoritmos

- Exemplo: Escrever um algoritmo para determinar se um número é maior que zero.

Ler Numero

Se Numero > 0 , faça:

Escrever “Maior que zero”

Senão, faça:

Escrever “Não é maior que zero”

Algoritmos

- Exemplo: Escrever um algoritmo para determinar se um número é zero, maior que zero ou menor que zero.

Ler Numero

Se Numero = 0, faça:

Escrever “Vale zero 10”

Senão se numero < 0, faça:

Escrever “Menor que zero”

Senão, faça:

Escrever “Maior que zero”

Algoritmos

- Exercício: Escrever um algoritmo que solicite uma base e uma altura. A seguir pergunte se é um triângulo ou um retângulo e informe a área da figura geométrica.

Escrever “Entre com a base”

Ler Base

Escrever “Entre com a altura”

Ler Altura

Escrever “Entre com T para triângulo ou R para retângulo”

Ler Resposta

Se Resposta = “T”, faça:

 Faça $\text{Area} \leftarrow (\text{Base} * \text{Altura}) / 2$

 Escrever “A área do triângulo vale”, Area

Senão se Resposta = “R”, faça:

 Faça $\text{Area} \leftarrow (\text{Base} * \text{Altura})$

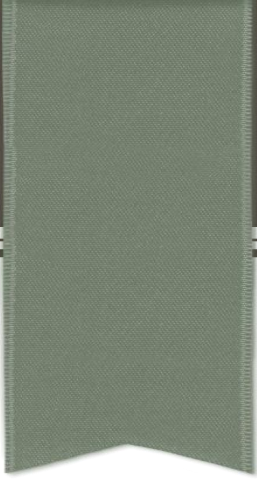
 Escrever “A área do retângulo vale”, Area

Senão, faça:

 Escrever “Não foi informada corretamente a figura geométrica”

Algoritmos

- Homework: Escrever um algoritmo que pergunte quantos membros uma família possui. Depois leia a idade de cada membro e no final informe a idade média da família.



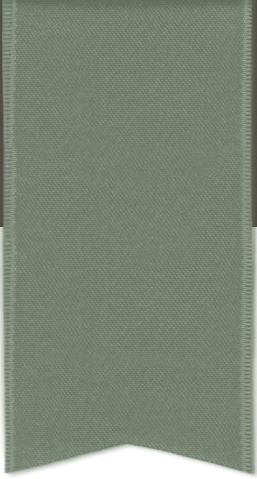
VARIÁVEIS

$x = y$

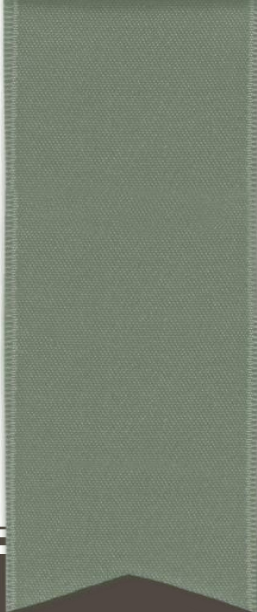
Variáveis

- Toda informação em um programa está armazenada na memória.
- Dados em particular são guardados em variáveis.
- Na computação uma **variável** é uma **posição de memória em que podemos guardar determinado dado ou valor e modificá-lo ao longo da execução do programa.**

MEMÓRIA		
ENDEREÇO	VARIÁVEL	CONTEÚDO
119		
120		
121	int x	10
122		
123		
124		



USANDO VARIÁVEIS



DECLARANDO UMA VARIÁVEL

Declarando uma variável

- Em linguagem C a declaração de uma variável segue a forma geral:

`tipo_da_variável nome_da_variável;`

- O `tipo_da_variável` determina o conjunto de valores e de operações que uma variável aceita.
- O `nome_da_variável` é como a variável é identificada no programa.
- Ao nome da variável o computador associa o endereço do espaço que ele reservou na memória para guardá-la.

Declarando uma variável

- Para declarar uma variável do tipo inteiro com nome x:

```
int x;
```

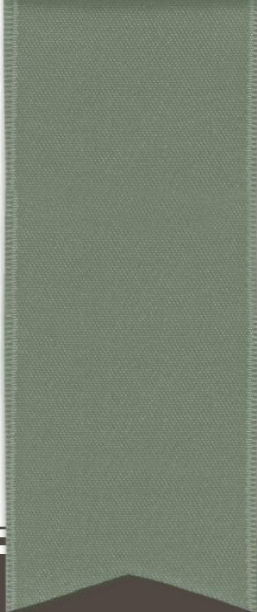
- Mais de uma variável pode ser declarada para um mesmo tipo ao mesmo tempo.
 - Basta separar cada nome de variável por uma vírgula.

```
int x,y,z;
```

- Toda declaração de variáveis termina com “;”.
- O ponto e vírgula separa as instruções da linguagem C.

Declarando uma variável

- Uma variável deve sempre ser declarada antes de ser usada.
 - Apenas quando uma variável é declarada que o computador reserva um espaço de memória para ela.
- Quando um espaço em memória é alocado para uma variável ela assume o valor que está naquele espaço.
 - A variável é inicializada com um “lixo de memória”.



NOMES DE VARIÁVEIS

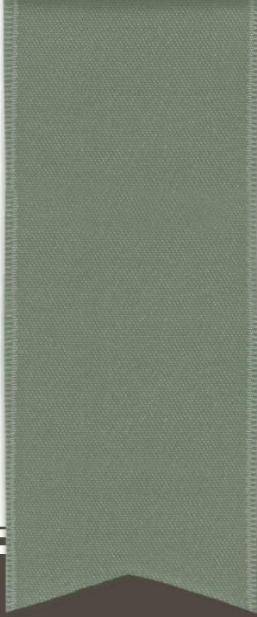
Nomes de variáveis

- O nome de uma variável é associado ao endereço reservado para ela na memória.
- Interessa ao programador saber apenas os nomes das variáveis.
- O nome de uma variável é um conjunto de caracteres que podem ser letras, números ou *underscores* (_).
- O nome de uma variável deve sempre iniciar com uma letra ou o *underscore* (_).
- A linguagem C é *case-sensitive*.
 - Uma palavra escrita utilizando caracteres maiúsculos é diferente da mesma palavra escrita com caracteres minúsculos.

Nomes de variáveis

- **Palavras-chave** não podem ser usadas como nome de variáveis.
- As palavras-chave são um conjunto de 32 palavras reservadas dentro da linguagem C.
 - Formam a sintaxe da linguagem de programação C.
 - Já possuem funções específicas dentro da linguagem.

Lista de palavras-chave da linguagem C							
auto	double	int	struct	break	else	long	switch
case	enum	if	typeof	continue	float	return	while
union	const	for	short	unsigned	char	extern	signed
void	default	do	sizeof	volatile	goto	register	static



TIPOS DE VARIÁVEIS

Tipos de variáveis

- A linguagem C possui um total de cinco tipos de dados básicos.

Tipo	Bits	Intervalo de valores
char	8	-128 A 127
int	32	-2.147.483.648 A 2.147.483.647
float	32	1,175494E-038 A 3,402823E+038
double	64	2,225074E-308 A 1,797693E+308
void	8	sem valor

Tipos de variáveis

char

- Permite armazenar em um único byte (8 bits) um número inteiro muito pequeno ou o código de um caractere do conjunto de caracteres da tabela ASCII:

```
char c = 'a';  
char n = 10;
```

- Caracteres sempre ficam entre aspas simples.
 - As aspas simples permitem que o compilador saiba que estamos inicializando nossa variável com um caractere e não com o conteúdo de outra variável.

Tipos de variáveis

int

- Permite armazenar um número inteiro.
- Seu tamanho depende do processador em que o programa está executando.
 - Tipicamente 16 ou 32 bits.

```
int n = 1459;
```

- Na linguagem C os tipos **char** e **int** podem ser especificados nas base:
 - Decimal → Base padrão.
 - Exemplo: **int x = 22231;**
 - Octal → Números precedidos por 0.
 - Exemplo: **int x = 053327;**
 - Hexadecimal → Números precedidos por 0x ou 0X.
 - Exemplo: **int x = 0x56D7;**

Tipos de variáveis

float e double

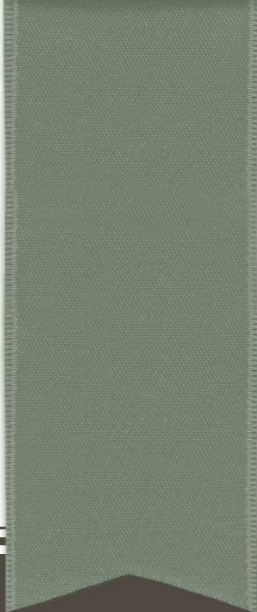
- Permite armazenar números reais (ou números de ponto flutuante).
- Tipo float → Precisão simples.
- Tipo double → Precisão dupla.
- São capazes de armazenar números muito grandes.
- A linguagem C usa o padrão numérico americano.
 - A parte decimal fica depois de um ponto.

```
float f = 5.25;  
double d = 15.673;
```

Tipos de variáveis

void

- Permite declarar uma função que não retorna valor.
- Permite também declarar um ponteiro genérico.
- A linguagem C não permite que se declare uma variável do tipo void.



MODIFICADORES DE TIPOS DE VARIÁVEIS

Modificadores de tipos de variáveis

- Além dos tipos básicos a linguagem C possui quatro modificadores de tipos.
- São aplicados precedendo os tipos básicos (com exceção do tipo void).
- Permitem adequar o tipo às necessidades do programa.
- São eles:
 - signed
 - unsigned
 - short
 - long

Modificadores de tipos de variáveis

signed

- Determina que uma variável declarada dos tipos char ou int poderá ter valores positivos ou negativos.
- É o modo padrão de definição de variáveis desses tipos.
 - Raramente é usado.

```
signed char x;  
signed int y;
```

Modificadores de tipos de variáveis

unsigned

- Determina que uma variável declarada dos tipos char ou int somente poderá ter valores positivos e o valor zero.
- Dobra a sua capacidade de armazenamento para valores positivos.
 - Uma variável do tipo **char** é capaz de armazenar valores de -128 até 127.
 - Uma variável do tipo **unsigned char** é capaz de armazenar valores de 0 até 255.

```
unsigned char x;  
unsigned int y;
```

Modificadores de tipos de variáveis

short

- Determina que uma variável do tipo **int** terá apenas 16 bits (inteiro pequeno), independentemente do processador.

```
short int i;
```

long

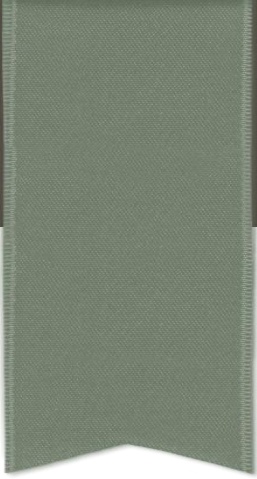
- Determina que uma variável do tipo **int** terá 32 bits, independentemente do processador.
- Também determina que o tipo **double** possua maior precisão.

```
long int n;  
long double d;
```

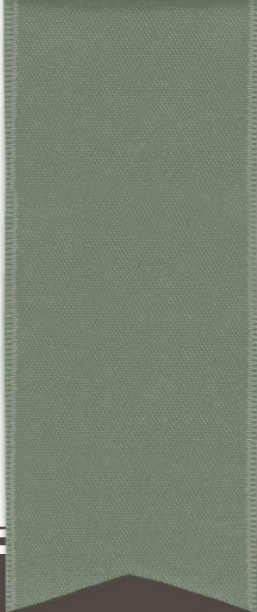
Modificadores de tipos de variáveis

- A linguagem C permite que se utilize mais de um modificador de tipo sobre um mesmo tipo.

Tipo	Bits	Intervalo de valores
char	8	-128 A 127
unsigned char	8	0 A 255
signed char	8	-128 A 127
int	32	-2.147.483.648 A 2.147.483.647
unsigned int	32	0 A 4.294.967.295
signed int	32	-32.768 A 32.767
short int	16	-32.768 A 32.767
unsigned short int	16	0 A 65.535
signed short int	16	-32.768 A 32.767
long int	32	-2.147.483.648 A 2.147.483.647
unsigned long int	32	0 A 4.294.967.295
signed long int	32	-2.147.483.648 A 2.147.483.647
float	32	1,175494E-038 A 3,402823E+038
double	64	2,225074E-308 A 1,797693E+308
long double	96	3,4E-4932 A 3,4E+4932



MOSTRANDO O CONTEÚDO DE VARIÁVEIS



PRINTF

printf

- A função `printf()` é uma das funções de saída de dados da linguagem C.
- Escreve na saída um conjunto de valores, caracteres e/ou sequência de caracteres de acordo com o formato especificado.
- Sua forma geral é:

`printf("tipos de saída", lista de variáveis);`

- Tipos de saída → Conjunto de caracteres que especifica o formato dos dados a serem escritos.
- Lista de variáveis → Conjunto de nomes de variáveis, separados por vírgula, que serão escritos.

printf

- A forma geral da função printf() especifica que ela receberá uma lista de variáveis para formatar e escrever na tela.
- Porém a função printf() pode ser usada para escrever apenas uma mensagem de texto simples na tela.

```
printf("Bem-vindo.");
```

printf

Escrevendo valores formatados

- Cada tipo de saída é precedido por um sinal de %, e um tipo de saída deve ser especificado para cada variável a ser escrita.
- Para escrever uma única expressão com o comando printf():

```
printf("%tipo", expressão);
```

- Para escrever duas expressões:

```
printf("%tipo1 %tipo2", expressão1, expressão2);
```

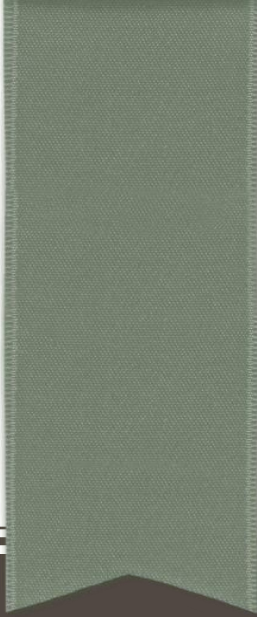
- e assim por diante.

printf

- A função `printf()` pode ser usada para escrever praticamente qualquer tipo de dado.

Alguns tipos de saída	
<code>%c</code>	escrita de um caractere (char)
<code>%d</code> ou <code>%i</code>	escrita de números inteiros (int ou char)
<code>%u</code>	escrita de números inteiros sem sinal (unsigned)
<code>%f</code>	escrita de número reais (float ou double)
<code>%s</code>	escrita de vários caracteres (string)
<code>%p</code>	escrita de um endereço de memória
<code>%e</code> ou <code>%E</code>	escrita em notação científica

```
printf("%s tem %d anos e %f metros de altura", nome, idade, altura);
```



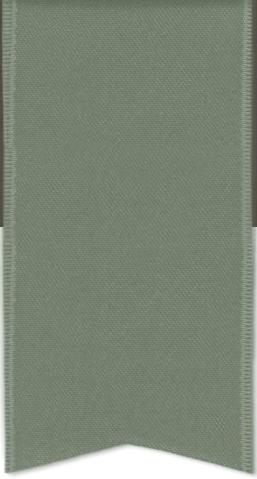
PUTCHAR

putchar

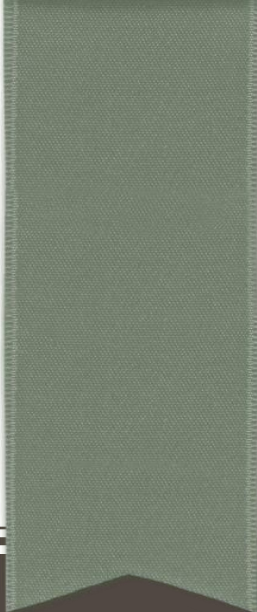
- A função **putchar()** permite escrever um único caractere na tela.
- Sua forma geral é:

int putchar(int caractere) ;

- A função **putchar()** recebe como parâmetro de entrada um valor inteiro.
 - Esse valor é convertido em caractere e mostrado na tela. A função retorna:
- O valor retornado pela função será:
 - O próprio caractere que foi escrito se não houver erro.
 - A constante EOF em caso de erro.



LENDO VALORES PARA VARIÁVEIS



SCANF

scanf

- A função **scanf()** é uma das funções de entrada de dados da linguagem C.
- A função **scanf()** lê do teclado um conjunto de valores, caracteres e/ou sequência de caracteres de acordo com o formato especificado.
- A forma geral da função scanf() é:

scanf("tipos de entrada", lista de variáveis);

- Tipos de entrada → Conjunto de caracteres que especifica o formato dos dados a serem lidos.
- Lista de variáveis → Conjunto de nomes de variáveis que serão lidos e separados por vírgula. Cada nome de variável é precedido pelo operador **&**.

scanf

- Os tipos de entrada especificam o formato de entrada dos dados que serão lidos.
- Cada tipo de entrada é precedido por um sinal de %.
 - Um tipo de entrada deve ser especificado para cada variável a ser lida.
- Para ler uma única variável com o comando scanf():

```
scanf("%tipo_de_entrada", &variável);
```

- Para ler duas variáveis:

```
scanf("%tipo1%tipo2", &var1, &var2);
```

- e assim por diante.

scanf

- Alguns tipos suportados pela linguagem C.

Alguns tipos de saída	
%c	leitura de um caractere (char)
%d ou %i	leitura de números inteiros (int ou char)
%f	leitura de números reais (float)
%s	leitura de vários caracteres (string)
%lf	leitura de números reais (double)

scanf

- Os comandos

```
scanf ("%d%d" , &x , &z) ;
```

- e

```
scanf ("%d %d" , &x , &z) ;
```

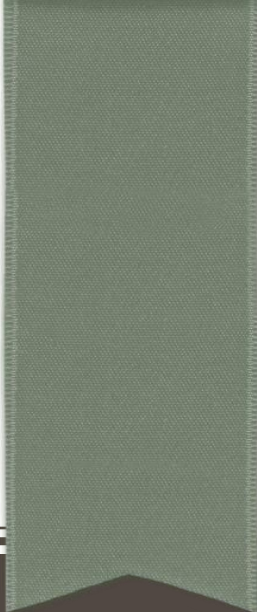
- são equivalentes.
 - O comando scanf() ignora os espaços em branco entre os tipos de entrada.
- Quando o comando scanf() é usado para ler dois ou mais valores, pode-se optar por duas formas de digitar os dados no teclado:
 - Digitar um valor e pressionar a tecla ENTER. Fazer isso para cada valor a ser digitado.
 - Digitar todos os valores separados por espaço e, por último, pressionar a tecla ENTER.

scanf

- Exemplo:

```
scanf ("%d/%d/%d", &dia, &mes, &ano) ;
```

- Permite que o comando scanf() seja usado para receber dados formatados como uma data: dia/mês/ano.
- Quando o usuário for digitar os três valores, será obrigado a digitar os três valores separados por barra.
 - As barras serão descartadas e não interferem nos dados.



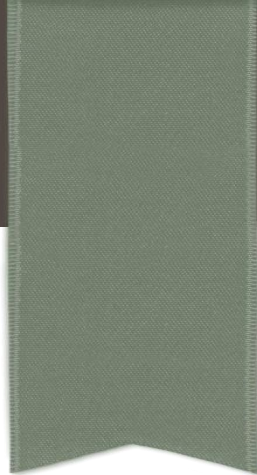
GETCHAR

getchar

- A função `getchar()` permite ler um único caractere do teclado.
- Sua forma geral é:

```
int getchar(void) ;
```

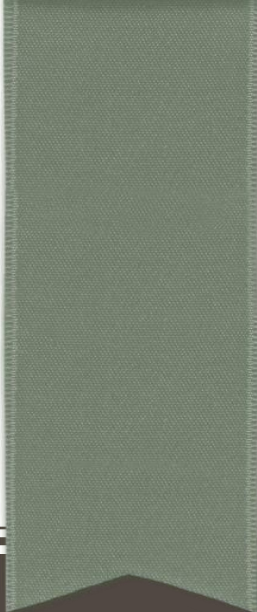
- Não recebe parâmetros de entrada.
- A função retorna:
 - O código ASCII do caractere lido se tudo correr bem.
 - A constante EOF caso algo aconteça de errado.



ESCOPO

Escopo

- O **escopo** é o conjunto de regras que determinam o uso e a validade das variáveis ao longo do programa.
- O escopo de uma variável **define onde e quando a variável pode ser usada.**
- Está ligado ao local de declaração da variável.
- Pode ser **global** ou **local**.



ESCOPO GLOBAL

Escopo global

- Uma variável declarada no **escopo global**, ou simplesmente **variável global**, é uma variável declarada fora de todas as funções do programa.
- É declarada na **área de declarações globais** do programa.
 - Antes das declarações das funções.
- Variáveis globais existirão enquanto o programa estiver executando.
- O tempo de vida de uma variável global é o tempo de execução do programa.
- Além disso, essas variáveis **podem ser acessadas e alteradas em qualquer parte do programa**.

Escopo global

```
#include <stdio.h>

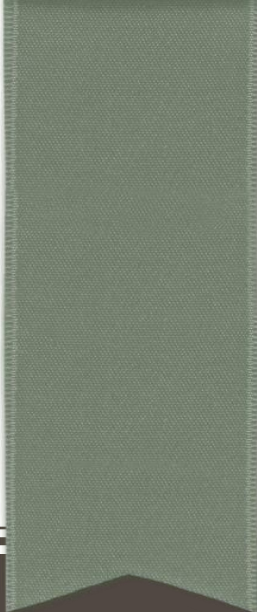
int x=5; // Variavel global

void incrementa(void) {
    x = x + 1;
}

int main(void) {
    printf("x = %d\n", x);
    incrementa();
    printf("x = %d\n", x);
    return 0;
}
```

Escopo global

- Variáveis globais devem ser evitadas sempre que possível.
- Qualquer parte do programa pode alterá-la, prejudicando a manutenção.
 - Torna mais difícil saber onde a variável é inicializada, para que serve, etc.
- Além disso variáveis globais ocupam memória durante todo o tempo de execução do programa, não apenas quando são necessárias.



ESCOPO LOCAL

Escopo local

- Uma variável declarada no **escopo local**, ou simplesmente **variável local**, é uma variável declarada dentro de um bloco de comandos delimitado por chaves ({ }).
- Essas variáveis são visíveis apenas no interior do bloco de comandos onde foram declaradas.
 - São válidas apenas dentro do seu escopo.
- No exemplo a variável x é declarada três vezes.
 - Cada uma em um bloco de comandos distinto.
 - Apesar de possuírem o mesmo nome possuem escopos diferentes.
 - Uma não existe enquanto a outra existir.

```
#include <stdio.h>

void funcao1(void) {
    int x; // Variavel local
}

void funcao2(void) {
    int x; // Variavel local
}

int main(void) {
    int x; // Variavel local
    scanf("%d", &x);
    if (x > 10) {
        int y=1; // Variavel local
        printf("%d", x);
    }
    return 0;
}
```

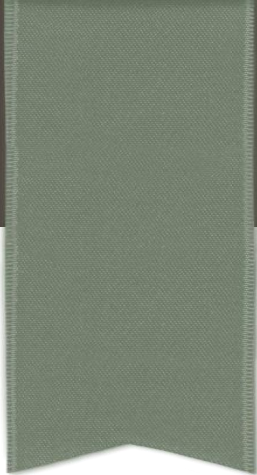
Escopo local

- Quando um bloco possuir uma variável local com o mesmo nome de uma variável global, **valerá a variável local**.
- Quando houver duas variáveis locais em blocos diferentes **valerá sempre a declaração mais próxima**.

```
#include <stdio.h>

int x=1; // Variavel global

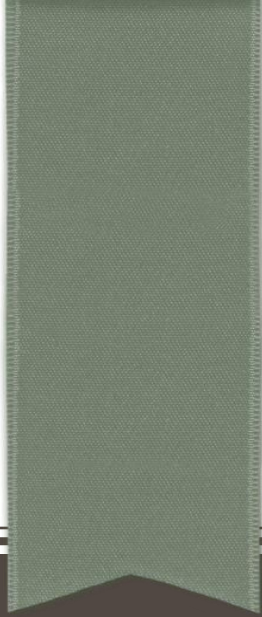
int main(void) {
    int x=2; // Variavel local
    if (x > 0) {
        int x=3; // Variavel local
        printf("%d\n", x); // Imprime 3
    }
    printf("%d\n", x); // Imprime 2
    return 0;
}
```



CONSTANTES

Constantes

- Uma **variável** é uma posição de memória na qual podemos guardar determinado dado ou valor.
 - **Podemos modificá-lo** ao longo da execução do programa.
- Uma **constante** permite guardar determinado dado ou valor na memória do computador, mas ele **não se alterará durante a execução do programa**.
- Para constantes é obrigatória a atribuição do valor no momento da declaração.
- Na linguagem C existem duas maneiras de criar constantes:
 - **#define**
 - **const**
- Existem também algumas constantes predefinidas.



VALOR LITERAL

Valor literal

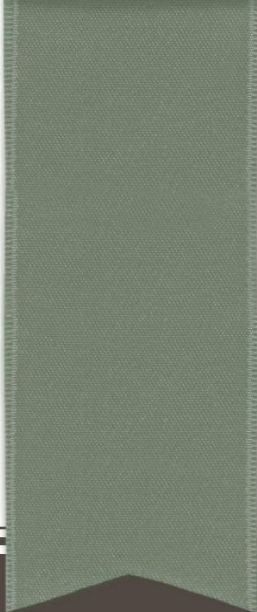
- Um **valor literal** é a representação de um dado dentro do próprio código-fonte.
- É um valor que é expresso como ele mesmo e não como um valor armazenado em variável ou o resultado de uma expressão aritmética.
- Na declaração a seguir, 10 é um valor literal:

```
int n = 10;
```

Valor literal

- A linguagem C permite que indiquemos o tipo de um valor literal pela adição de um sufixo.
- Um sufixo é um caractere de tipo acrescentado ao final do valor literal, sem nenhum espaço entre eles.

Sufixo	Tipo	Exemplo
"sem sufixo"	double	10.5
	int	10
"f" ou "F"	float	3.14F
"l" ou "L"	long double	123456.789L
	long int	123456799L
"u" ou "U"	unsigned int	453U
"ul" ou "UL"	unsigned long int	123456789UL



O COMANDO #DEFINE

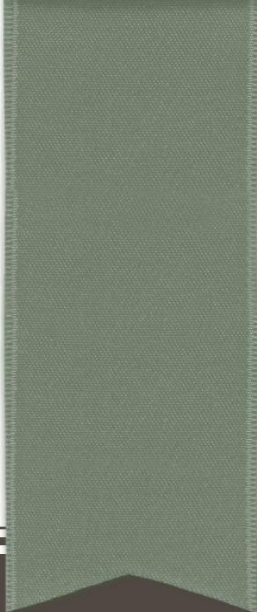
O comando `#define`

- O comando **`#define`** segue a forma geral:

`#define nome_da_constante valor_da_constante`

- O comando **`#define`** é uma diretiva de compilação.
 - Informa ao compilador que ele deve procurar todas as ocorrências da palavra definida por **`nome_da_constante`** e substituir por **`valor_da_constante`** quando o programa for compilado.
- Exemplo:

`#define PI 3.1415`



O COMANDO CONST

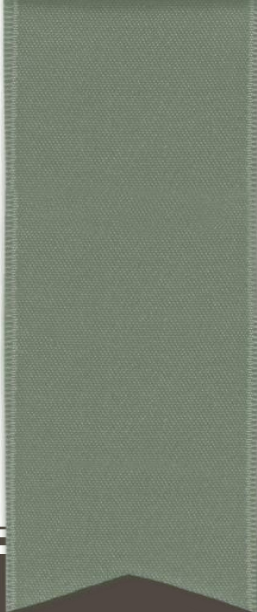
O comando const

- O comando const segue a forma geral:

```
const tipo nome_da_constante = valor_da_constante;
```

- A geral se parece muito com a da declaração de uma variável.
- A diferença é o prefixo **const**, que informa ao programa que a variável declarada não poderá ter seu valor alterado.
- Por ser uma variável, essa constante está sujeita às mesmas regras que regem o uso das variáveis.
- Exemplo:

```
const float PI = 3.1415;
```



SEQUÊNCIAS DE ESCAPE

Sequências de escape

- A linguagem C possui algumas constantes predefinidas, como
- As **sequências de escape** (ou códigos de barra invertida) são constantes predefinidas que **permitem o envio de caracteres de controle** não gráficos para dispositivos de saída.

Código	Comando
\a	som de alerta (bip)
\b	retrocesso (backspace)
\n	nova linha (new line)
\r	retorno de carro (carriage return)
\v	tabulação vertical
\t	tabulação horizontal
\'	apóstrofe
\"	aspa
\\	barra invertida (backslash)
\f	alimentação de folha (form feed)
\?	símbolo de interrogação
\0	caractere nulo (cancela a escrita do restante)

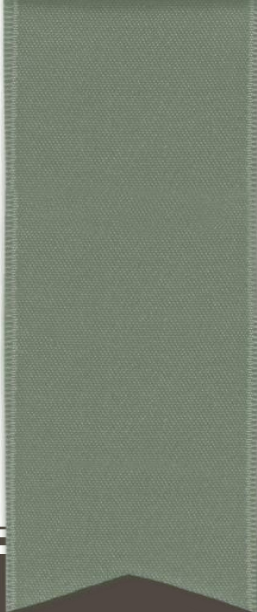
Sequências de escape

```
#include <stdio.h>

int main(void) {
    printf("Ola mundo!\nOla\nmundo!\n");
    printf("Ola \\ mundo!\n\"Ola mundo!\"\n");
    return 0;
}
```

- A execução do programa gerará como saída:

```
Ola mundo!
Ola
mundo!
Ola \ mundo!
"Ola mundo!"
```



EXERCÍCIOS

Exercícios

1. Faça um programa que leia 2 números e depois imprima as 4 operações matemáticas sobre estes 2 números (soma, subtração, multiplicação e divisão). Os resultados devem aparecer um por linha e deve haver apenas uma instrução **printf()** para mostrar os resultados das operações.
2. Faça um programa que solicite o dia, o mês e o ano e depois imprima no formato “dia\mês\ano” (utilizando a barra invertida).
3. Faça um programa que pergunte o raio de um círculo e mostre sua área. Faça uma versão com π sendo uma constante definida por **#define** e outra definida por **const**.
 - Área = πr^2 , sendo r o raio do círculo.
 - Considere $\pi = 3,1415$.

Exercícios

- Resposta do exercício 1: Faça um programa que leia 2 números e depois imprima as 4 operações matemáticas sobre estes 2 números (soma, subtração, multiplicação e divisão). Os resultados devem aparecer um por linha e deve haver apenas uma instrução **printf()** para mostrar os resultados das operações.

```
#include <stdio.h>
#include <locale.h>

int main(void) {
    float a,b;
    setlocale(LC_ALL,"Portuguese");
    printf("Entre com o primeiro número: ");
    scanf("%f", &a);
    printf("Entre com o segundo número: ");
    scanf("%f", &b);
    printf("Soma: %f\nSubtração: %f\nMultiplicação: %f\nDivisão: %f\n", a+b, a-b, a*b, a/b);
    return 0;
}
```

Exercícios

- Resposta do exercício 2: Faça um programa que solicite o dia, o mês e o ano e depois imprima no formato “dia\mês\ano” (utilizando a barra invertida).

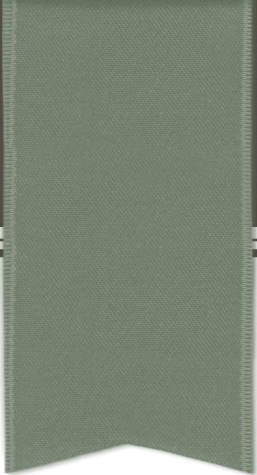
```
#include <stdio.h>
#include <locale.h>
int main(void) {
    int dia, mes, ano;
    setlocale(LC_ALL, "Portuguese");
    printf("Entre com o dia: ");
    scanf("%d", &dia);
    printf("Entre com o mês: ");
    scanf("%d", &mes);
    printf("Entre com o ano: ");
    scanf("%d", &ano);
    printf("Data: %d\\%d\\%d", dia, mes, ano);
    return 0;
}
```

Exercícios

- Resposta do exercício 3: Faça um programa que pergunte o raio de um círculo e mostre sua área. Faça uma versão com π sendo uma constante definida por **#define** e outra definida por **const**.

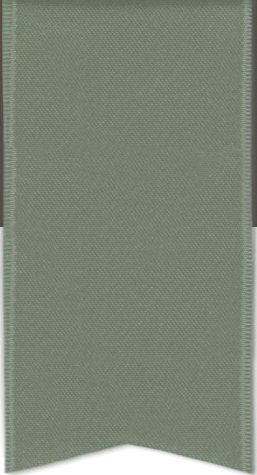
```
#include <stdio.h>
#include <locale.h>
#define pi 3.1415
int main(void) {
    float raio, area;
    setlocale(LC_ALL, "Portuguese");
    printf("Entre com o raio: ");
    scanf("%f", &raio);
    area = pi * raio * raio;
    printf("A área vale: %f\n", area);
    return 0;
}
```

```
#include <stdio.h>
#include <locale.h>
int main(void) {
    const float pi=3.1415;
    float raio, area;
    setlocale(LC_ALL, "Portuguese");
    printf("Entre com o raio: ");
    scanf("%f", &raio);
    area = pi * raio * raio;
    printf("A área vale: %f\n", area);
    return 0;
}
```



OPERAÇÕES SOBRE VARIÁVEIS

$$x = y^2 - 3$$



OPERADOR DE ATRIBUIÇÃO

Operador de atribuição

- Uma das operações mais utilizadas em programação é a operação de **atribuição**.
- Em linguagem C a atribuição é realizada pelo operador “**=**”.
- Armazena determinado valor em uma variável.
- Segue esta forma geral:

nome_da_variável = expressão;

- O operador de atribuição também pode ser utilizado no momento da declaração da variável.
 - A variável já é declarada possuindo um valor inicial.

Operador de atribuição

- A linguagem C permite a atribuição entre tipos básicos diferentes.
 - O compilador **converte automaticamente o valor**.
- Durante a etapa de conversão de tipos **pode haver perda de informação**.
- O programa ao lado apresenta como saída:

c: A
i: 3
f: 65.000000
g: 3.000000

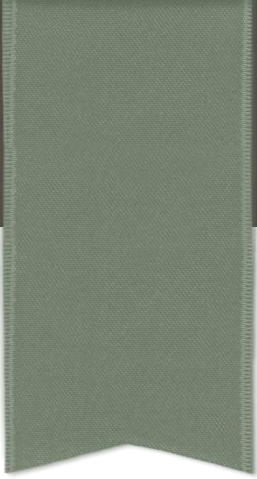
```
#include <stdio.h>
int main(void) {
    char c;
    int i=65;
    float f=3.1415, g;
    /* c recebe os 8 bits menos
    significativo de i */
    c = i;
    /* i recebe a parte inteira de f */
    i = f;
    /* r recebe a o valor de 8 bits
    convertido para float */
    f = c;
    /* g recebe o valor de i */
    g = i;
    printf("c: %c\ni: %d\nf: %f\ng: %f\n",
        c, i, f, g);
    return 0;
}
```

Operador de atribuição

- O operador de atribuição pode ser utilizado em cascata.

```
#include <stdio.h>

int main(void) {
    int a, b, c, d;
    a = b = (c = d = 1) + 4;
    printf("a: %d\nb: %d\nc: %d\nd: %d\n", a, b, c, d);
    return 0;
}
```

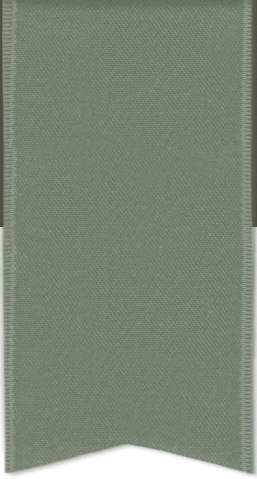


OPERADORES ARITMÉTICOS

Operadores aritméticos

- **Operadores aritméticos** são aqueles que operam sobre números (valores, variáveis, constantes ou chamadas de funções) e/ou expressões.
- Têm como resultados valores numéricos.
- As operações de multiplicação, divisão e resto tem **precedência** sobre as operações de adição e subtração.

Operador	Significado	Exemplo
+	adição de dois valores	$z = x + y$
-	subtração de dois valores	$z = x - y$
*	multiplicação de dois valores	$z = x * y$
/	quociente de dois valores	$z = x / y$
%	resto de uma divisão	$z = x \% y$



OPERADORES RELACIONAIS

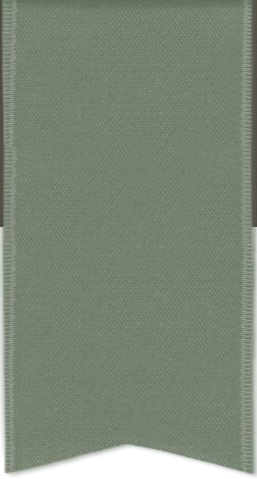
Operadores relacionais

- Os **operadores relacionais** são aqueles que operam sobre dois valores (valores, variáveis, constantes ou chamadas de funções) e/ou expressões e verificam a magnitude (qual é maior ou menor) e/ou igualdade entre eles.
- São operadores de comparação de valores.

Operador	Significado	Exemplo
>	Maior do que	$x > 5$
>=	Maior ou igual a	$x \geq 10$
<	Menor do que	$x < 5$
<=	Menor ou igual a	$x \leq 10$
==	Igual a	$x == 0$
!=	Diferente de	$x != 0$

Operadores relacionais

- Como resultado um operador relacional retorna:
 - O **valor 1** se a expressão relacional for **verdadeira**.
 - O **valor 0** se a expressão relacional for **falsa**.
- Não confundir o operador de atribuição “=” com o operador de comparação “==”.



OPERADORES LÓGICOS

Operadores lógicos

- Certas situações não podem ser modeladas apenas utilizando os operadores aritméticos e/ou relacionais.
- Exemplo: saber se determinada variável x está dentro de uma faixa de valores.
 - $0 < x < 10$
- Para modelar esse tipo de situação a linguagem C possui um conjunto de operadores lógicos.

Operador	Significado	Exemplo
&&	Operador E	$(x \geq 0 \ \&\& \ x \leq 9)$
	Operador OU	$(a == 'F' \ \ b != 32)$
!	Operador NEGAÇÃO	$!(x == 10)$

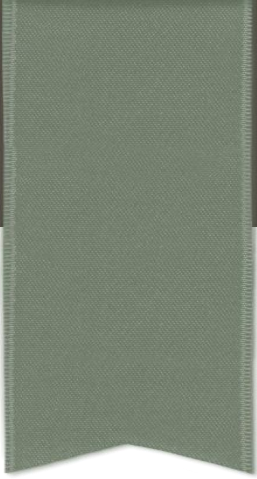
Operadores lógicos

- Permitem representar situações lógicas unindo duas ou mais expressões relacionais simples em uma composta:
 - **Operador E (&&)**
 - A expressão resultante só é verdadeira se ambas as expressões unidas por esse operador também forem verdadeiras.
 - Exemplo: $(x \geq 0 \ \&\& \ x \leq 9)$ será verdadeira somente se as expressões $(x \geq 0)$ e $(x \leq 9)$ forem verdadeiras.
 - **Operador OU (||)**
 - A expressão resultante é verdadeira se alguma das expressões unidas por esse operador também for.
 - Exemplo: $(a == 'F' \ || \ b != 32)$ será verdadeira se uma de suas duas expressões, $(a == 'F')$ ou $(b != 32)$, for verdadeira.
 - **Operador NEGAÇÃO (!)**
 - Inverte o valor lógico da expressão à qual se aplica.
 - Exemplo: $!(x == 10)$ será verdadeira de x for diferente de 10.
- Os operadores lógicos atuam sobre valores lógicos e retornam um valor lógico.
 - **1** → Se a expressão é **verdadeira**.
 - **0** → Se a expressão é **falsa**.

Operadores lógicos

- Tabela verdade:

Tabela-verdade					
a	b	!a	!b	a && b	a b
0	0	1	1	0	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	1	1



OPERADORES BIT A BIT

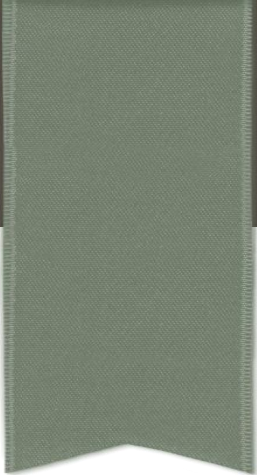
Operadores bit a bit

- A linguagem C permite que se façam operações “bit a bit” em valores numéricos.
- Os operadores bit a bit ajudam os programadores que necessitam trabalhar em “baixo nível”.

Operador	Significado	Exemplo
~	complemento bit a bit	~x
&	E bit a bit	x & 167
	OU bit a bit	x 129
^	OU exclusivo	x ^ 167
<<	deslocamento de bits à esquerda	x << 2
>>	deslocamento de bits à direita	x >> 2

Operadores bit a bit

- Os operadores bit a bit só podem ser usados nos tipos char, int e long.

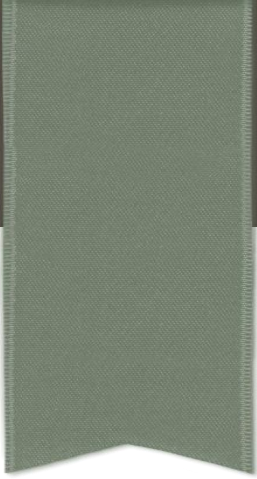


OPERADORES DE ATRIBUIÇÃO SIMPLIFICADA

Operadores de atribuição simplificada

- A linguagem C permite simplificar algumas expressões.

Operador	Significado	Exemplo		
+=	soma e atribui	x += y	igual	x = x + y
-=	subtrai e atribui	x -= y	igual	x = x - y
*=	multiplica e atribui	x *= y	igual	x = x * y
/=	divide e atribui quociente	x /= y	igual	x = x / y
%=	divide e atribui resto	x %= y	igual	x = x % y
&=	E bit a bit e atribui	x &= y	igual	x = x & y
=	OU bit a bit e atribui	x = y	igual	x = x y
^=	OU exclusivo e atribui	x ^= y	igual	x = x ^ y
<<=	desloca à esquerda e atribui	x <<= y	igual	x = x << y
>>=	desloca à direita e atribui	x >>= y	igual	x = x >> y



OPERADORES DE PRÉ E PÓS- INCREMENTO/DECREMENTO

Operadores de pré e pós-incremento/decremento

- A linguagem C possui operadores de incremento (++) e decremento (--).
 - Podem ser utilizados sempre que for necessário somar uma unidade (incremento) ou subtrair uma unidade (decremento).

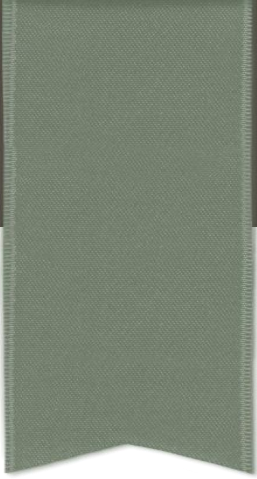
Operador	Significado	Exemplo	Resultado
++	incremento	++x ou x++	$x = x + 1$
--	decremento	--x ou x--	$x = x - 1$

Operadores de pré e pós-incremento/decremento

- Estes operadores possuem embutida uma operação de atribuição.
- Esse operador pode ser usado antes ou depois do nome da variável, com uma diferença significativa:
 - **++x (pré-incremento)** → Soma 1 à variável x **antes** de utilizar seu valor.
 - **x++ (pós-incremento)** → Soma 1 à variável x **depois** de utilizar seu valor.
 - **--x (pré-decremento)** → Subtrai 1 da variável x **antes** de utilizar seu valor.
 - **x-- (pós-decremento)** → Subtrai 1 da variável x **depois** de utilizar seu valor.

Operação	Equivale a
<code>a = ++b;</code>	<code>b = b + 1;</code> <code>a = b;</code>
<code>a = b++;</code>	<code>a = b;</code> <code>b = b + 1;</code>

Operação	Equivale a
<code>a = --b;</code>	<code>b = b - 1;</code> <code>a = b;</code>
<code>a = b--;</code>	<code>a = b;</code> <code>b = b - 1;</code>



MODELADORES DE TIPOS (*CASTS*)

Modeladores de tipos (*casts*)

- **Modeladores de tipos** (também chamados de *type cast*) são uma forma explícita de conversão de tipo.
- O uso de um modelador de tipo segue a forma geral:

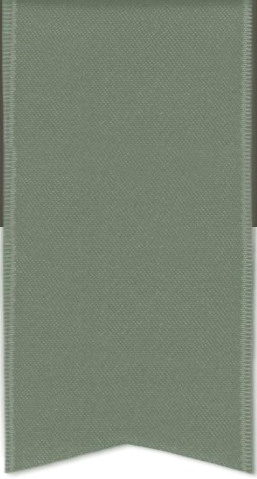
(nome_do_tipo) expressão

- Um modelador de tipo é definido pelo próprio **nome_do_tipo** entre parênteses.
- É colocado à frente de uma expressão e tem como objetivo forçar o resultado da expressão a ser de um tipo especificado.
- O resultado do programa ao lado será:

```
a: 2.310000  
b: 2.000000
```

```
#include <stdio.h>

int main(void) {
    float a, b, f=23.1;
    a = f/10;
    b = (int) (f/10);
    printf("a: %f\nb: %f\n", a, b);
    return 0;
}
```



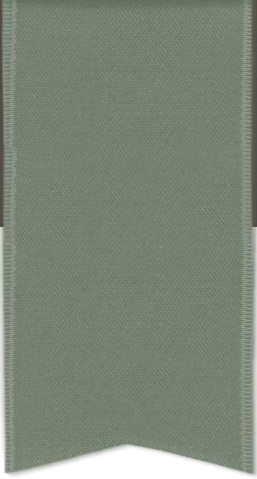
OPERADOR VÍRGULA (,)

Operador vírgula (,)

- O operador vírgula (,) pode ser utilizado de duas maneiras:
 - Como pontuação.
 - Por exemplo, para separar parâmetros de uma função: `printf("a: %d\nb: %d\n", a, b);`
 - Para determinar uma lista de expressões que devem ser executadas sequencialmente.
 - Exemplo: `a = (b = 2, b + 3);`
 - No exemplo as expressões são executadas da esquerda para a direita.
 - O valor 2 é atribuído a b.
 - O valor 3 é somado a b e atribuído à variável a.
 - Pode-se encadear quantos operadores “,” forem necessários.
- O operador “,” é um separador de comandos, enquanto o operador “;” é um terminador de comandos.
- O resultado do programa ao lado será:
`a: 5`
`b: 2`

```
#include <stdio.h>

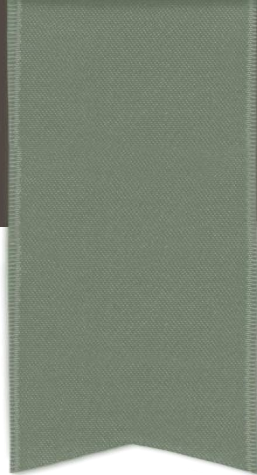
int main(void) {
    int a, b;
    a = (b = 2, b + 3);
    printf("a: %d\nb: %d\n", a, b);
    return 0;
}
```



PRECEDÊNCIA DE OPERADORES

Precedência de operadores

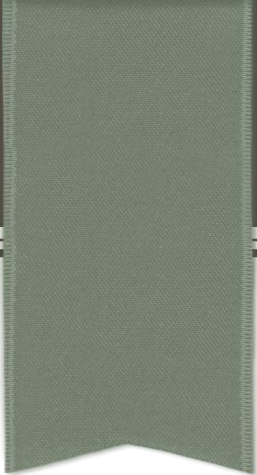
MAIOR PRECEDÊNCIA	
++ --	Pré-incremento/decremento
()	Parênteses (chamada de função)
[]	Elemento de array
.	Elemento de struct
->	Conteúdo de elemento de ponteiro para struct
++ --	Pós-incremento/decremento
+ -	Adição e subtração unária
! ~	Não lógico e complemento bit a bit
(tipo)	Conversão de tipos (<i>type cast</i>)
*	Acesso ao conteúdo de ponteiro
&	Endereço de memória do elemento
sizeof	Tamanho do elemento
* / %	Multiplicação, divisão e módulo (resto)
+ -	Adição e subtração
<< >>	Deslocamento de bits à esquerda e à direita
< <=	"Menor do que" e "menor ou igual a"
> >=	"Maior do que" e "maior ou igual a"
== !=	"Igual a" e "diferente de"
&	E bit a bit
^	OU exclusivo
	OU bit a bit
&&	E lógico
	OU lógico
?:	Operador ternário
=	Atribuição
+= -=	Atribuição por adição ou subtração
*= /= %=	Atribuição por multiplicação, divisão ou módulo (resto)
<<= >>=	Atribuição por deslocamento de bits
&= ^= =	Atribuição por operações lógicas
,	Operador vírgula
MENOR PRECEDÊNCIA	



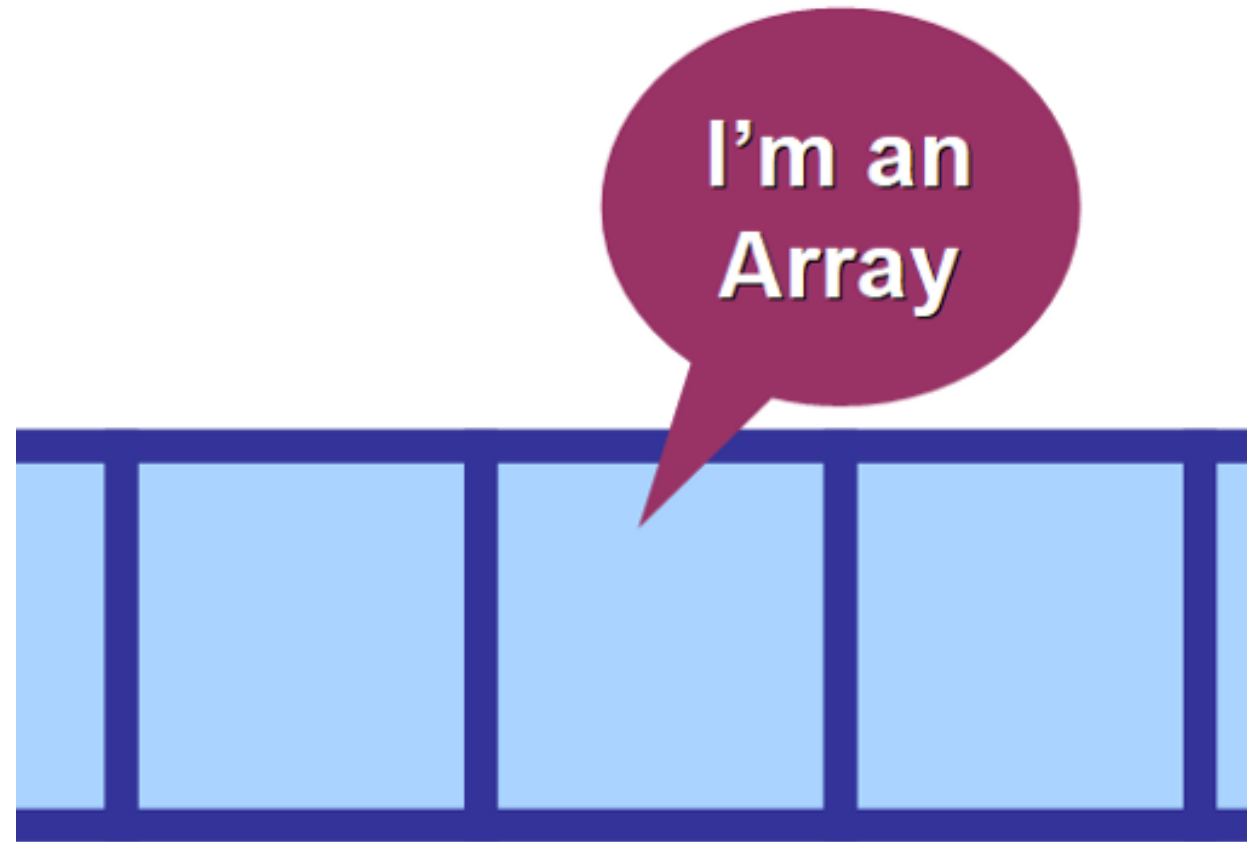
EXERCÍCIOS

Exercícios

1. Leia um número real e informe quanto vale 10% deste número.
2. Calcule o ano de nascimento de uma pessoa a partir de sua idade e do ano atual. Assumir que a pessoa já fez aniversário no ano.
3. Leia um valor em reais e a cotação do dólar. Em seguida, imprima o valor correspondente em dólares.
4. Leia um valor que represente uma temperatura em graus Celsius e apresente-a convertida em graus Fahrenheit. A fórmula de conversão é: $F = C * (9/5) + 32$, sendo F a temperatura em Fahrenheit e C a temperatura em Celsius.
5. Leia um número real e coloque-o com precisão de 2 casas decimais. Exemplo: se for fornecido o número 12,3456 o valor deverá ser 12,34.
6. Leia um número inteiro positivo de três dígitos e em seguida calcule e mostre o número formado pelos dígitos invertidos do número lido. Exemplo: se for lido o número 123 deve ser mostrado o número gerado = 321.



VETORES E MATRIZES (ARRAYS)



Vetores e matrizes (arrays)

- As variáveis utilizadas até o momento são capazes de armazenar um único valor por vez.
- Sempre que é atribuído um novo valor a uma variável o valor anterior é perdido.
- Isso ocorre porque cada variável está associada a uma única posição de memória.
 - Para armazenar mais de um valor, é preciso usar mais de uma variável.
- Imagine o seguinte problema: leia as notas de uma turma de cinco estudantes e depois imprima as notas que são maiores do que a média da turma.

Vetores e matrizes (arrays)

```
#include <stdio.h>

int main(void) {
    float media, n1, n2, n3, n4, n5;
    printf("Digite a nota de 5 estudantes: ");
    scanf("%f", &n1);
    scanf("%f", &n2);
    scanf("%f", &n3);
    scanf("%f", &n4);
    scanf("%f", &n5);
    media = (n1+n2+n3+n4+n5)/5;
    if (n1>media) printf("Nota: %f\n", n1);
    if (n2>media) printf("Nota: %f\n", n2);
    if (n3>media) printf("Nota: %f\n", n3);
    if (n4>media) printf("Nota: %f\n", n4);
    if (n5>media) printf("Nota: %f\n", n5);
    return 0;
}
```

Vetores e matrizes (arrays)

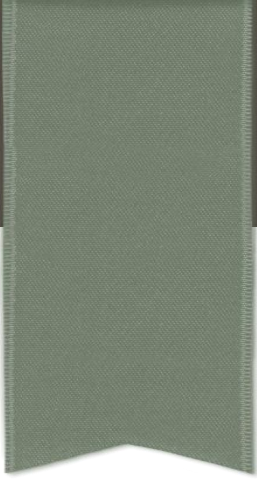
- O programa apresenta uma solução possível para o problema.
 - O inconveniente é a grande quantidade de variáveis para gerenciar e o uso repetitivo de comandos praticamente idênticos.
- A solução é inviável para uma turma de 100 alunos.
- Teríamos:
 - Uma variável para armazenar a nota de cada aluno: 100 variáveis.
 - Um comando de leitura para cada nota: 100 **scanf()**.
 - Um somatório de 100 notas.
 - Um comando de teste para cada aluno: 100 comandos **if**.
 - Um comando de impressão na tela para cada aluno: 100 **printf()**.

Vetores e matrizes (arrays)

- Essa linha apresenta uma solução extremamente rígida para o problema.
- Modificar a quantidade de alunos implica reescrever todo o código.
 - Repetir comandos praticamente idênticos.
- Além disso existe uma grande quantidade de variáveis para gerenciar.
 - Cada uma com o próprio nome.
 - Torna a tarefa ainda mais difícil de ser realizada.

Vetores e matrizes (arrays)

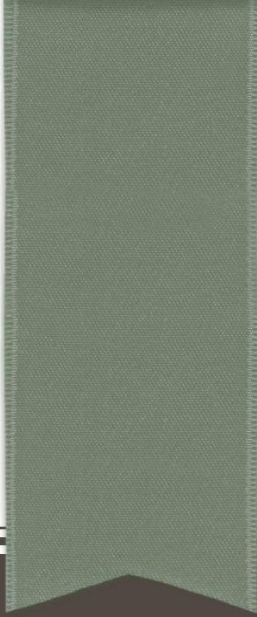
- Uma **solução melhor é a utilização de array**.
- Como as variáveis do programa armazenam notas de alunos, podemos declará-las usando um **nome único para todos os 100 alunos**.
- Um array, ou vetor, é a forma mais simples e comum de dados estruturados da linguagem C.
 - Um conjunto de variáveis de um mesmo tipo, todas associadas ao mesmo nome.
 - São acessíveis por um índice.



ARRAY COM UMA DIMENSÃO (VETOR)

Array com uma dimensão (vetor)

- A **ideia** de um array é bastante simples: **criar um conjunto de variáveis do mesmo tipo utilizando apenas um nome**.
- No caso do exemplo anterior, as notas dos 100 alunos são todas do mesmo tipo.
 - Essa solução permitiria usar apenas um nome de variável para representar todas as notas dos alunos.
- Um array com uma dimensão é conhecido como **array unidimensional**.



DECLARAÇÃO DE ARRAY UNIDIMENSIONAL

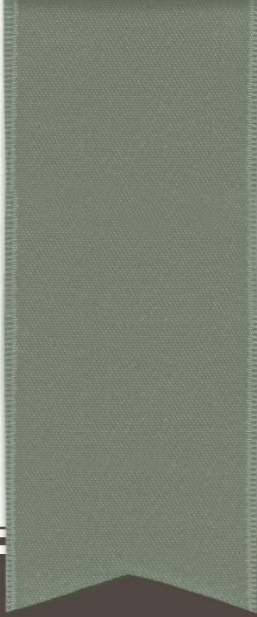
Declaração de array unidimensional

- A declaração de um array segue a forma geral:

tipo_dado nome_array[tamanho] ;

- Esse comando define um array de nome **nome_array** contendo **tamanho** elementos adjacentes na memória.
- **tamanho** deve ser sempre uma **constante inteira maior ou igual a zero**.
- Cada elemento do array é do tipo **tipo_dado**.
- Um array de números reais contendo 100 elementos seria declarado como:

float notas[100] ;



ACESSANDO UM ELEMENTO DO ARRAY

Acessando um elemento do array

- A variável nota possui o mesmo nome que as demais notas dos outros alunos.
 - O acesso ao valor de cada nota é feito utilizando um índice.

```
float notas[100];  
notas[0] = 81;  
notas[1] = 55;  
...  
notas[99] = 72;
```



- Na posição “0” do array, está armazenado o valor “81”, na posição “1” está armazenado o valor “55”, e assim por diante.

Acessando um elemento do array

- A leitura das notas pode ser realizada com um laço.
- É utilizado um comando de repetição (laço **for**) para executar o comando de leitura **scanf()** para ler cada uma das 100 notas e armazená-las no array.
- Como o array possui um índice, pode ser utilizado comandos de repetição para executar repetidamente um mesmo comando para cada posição do array.
- Torna o código mais limpo, facilitando sua manutenção e minimizando a chance de erros.

```
#include <stdio.h>

int main(void) {
    float notas[100];
    int i;
    for (i=0; i<100; i++) {
        printf("Entre com a nota %d: ", i);
        scanf("%f", &notas[i]);
    }
    return 0;
}
```

Acessando um elemento do array

- Cada posição do array possui todas as características de uma variável.
 - Ela pode aparecer em comandos de entrada e saída de dados, expressões e atribuições.

- Exemplo:

```
scanf ("%d", &notas[5]) ;           //comando de leitura
notas[0] = 10 ;                     //comando de atribuição
notas[1] = notas[5] + notas[0] ;    //expressão
```

- O tempo para acessar qualquer uma das posições do array é o mesmo.

Acessando um elemento do array

- Sendo N a quantidade de elementos definidos na declaração do array:
 - Na linguagem C a numeração do índice do array **começa sempre do ZERO** e **termina sempre em N-1**.
- Se o array foi declarado como
`float notas[100];`
- as notas serão indexadas de 0 a 99:
- Isso acontece porque **um array é um agrupamento de dados do mesmo tipo, adjacentes na memória**.
 - O nome do array indica onde esses dados começam na memória.
 - O índice do array indica quantas posições se deve pular para acessar determinada posição.

Acessando um elemento do array

MEMÓRIA		
ENDEREÇO	VARIÁVEL	CONTEÚDO
123	notas	81
124		55
125		63
126		67
127		90
128		
129		

notas[0]

notas[1]

notas[2]

notas[3]

notas[4]

Acessando um elemento do array

- Em um array de 100 elementos, índices menores do que 0 e maiores do que 99 também podem ser utilizados.
 - Pode resultar nos mais variados erros durante a execução do programa.
- Um array é um agrupamento de dados adjacentes na memória, e o seu índice apenas indica quantas posições se deve pular para acessar determinada posição.
 - Se usar o índice 100 o programa tentará acessar a centésima posição a partir da posição inicial.
 - Se usar o índice -1 o programa tentará acessar uma posição anterior ao local onde o array começa na memória.
- O problema é que essas posições existirem na memória e são acessíveis, mas elas não pertencem ao array.
- É função do programador garantir que os limites do array estejam sendo respeitados.

Acessando um elemento do array

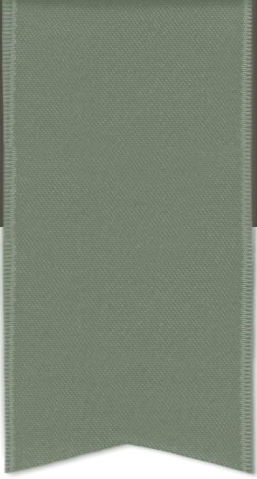
- O programa ao lado está errado.

```
#include <stdio.h>
int main(void) {
    int m[5] = {1, 2, 3, 4, 5}, n[5];
    n = m; // Erro
    return 0;
}
```



- A linguagem C não suporta a atribuição de um array para outro.
- Para atribuir o conteúdo de um array a outro array o correto é copiar seus valores, elemento por elemento.

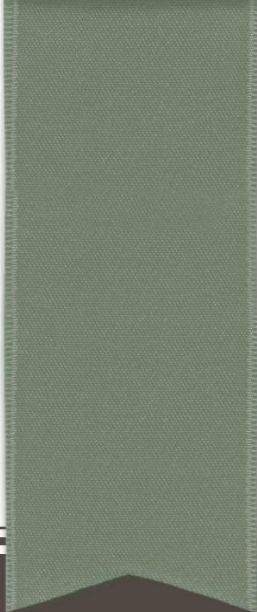
```
#include <stdio.h>
int main(void) {
    int m[5] = {1, 2, 3, 4, 5}, n[5], i;
    for (i=0; i<5; i++)
        n[i] = m[i];
    return 0;
}
```



ARRAY COM DUAS DIMENSÕES (MATRIZ)

Array com duas dimensões (matriz)

- Os arrays declarados até o momento são unidimensionais e foram tratados como uma lista de variáveis.
- Há casos em que uma estrutura com mais de uma dimensão é necessária.
 - Uma tabela, por exemplo.
- Para casos como estes são utilizados **arrays com duas dimensões** (arrays bidimensionais), ou seja, uma **matriz**.



DECLARAÇÃO DE ARRAY BIDIMENSIONAL (MATRIZ)

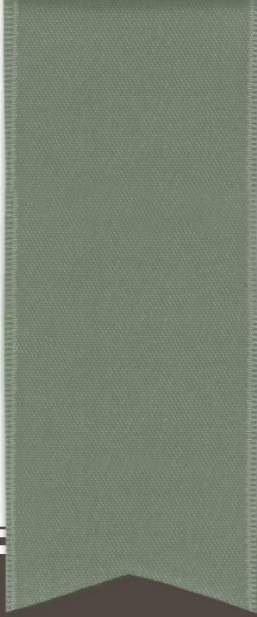
Declaração de array bidimensional (matriz)

- A declaração de uma matriz segue a forma geral:

```
tipo_dado nome_array[qtde_linhas][qtde_colunas];
```

- Define um array de nome **nome_array** contendo **qtde_linhas** x **qtde_colunas** elementos do mesmo tipo e adjacentes na memória.
- Cada elemento do array é do tipo **tipo_dado**.
- Exemplo: para criar um array de inteiros que possua 100 linhas e 50 colunas usa-se a declaração:

```
int mat[100][50];
```



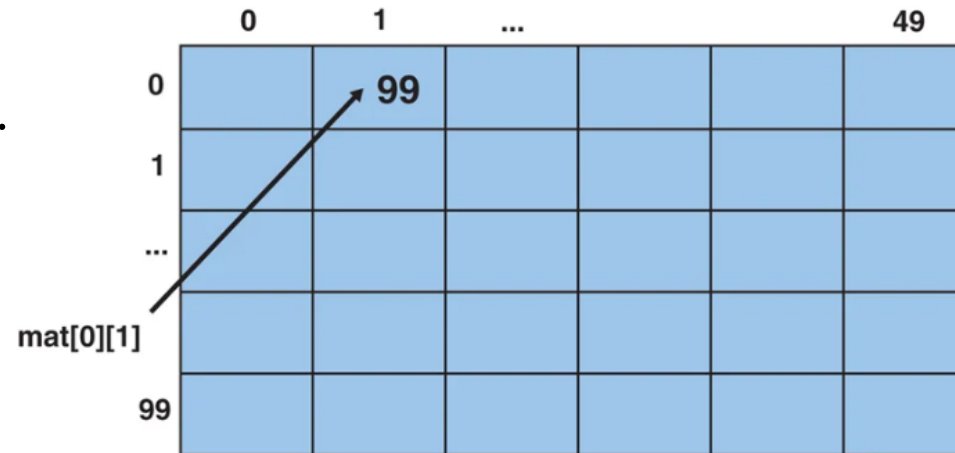
ACESSANDO UM ELEMENTO DO ARRAY BIDIMENSIONAL

Acessando um elemento do array bidimensional

- Uma matriz (array bidimensional) possui **duas dimensões: linhas e colunas**.
- Para acessar determinada posição da matriz é necessário utilizar dois índices:
 - O primeiro índice especifica a linha.
 - O segundo índice especifica a coluna.

- Exemplo:

```
int mat[100][50];  
mat[0][1] = 99;
```



- Cada índice possui o seu próprio par de colchetes.
- Cada dimensão começa no índice ZERO e termina em N-1.
 - N é a quantidade de elementos para aquela dimensão da matriz.

Acessando um elemento do array bidimensional

- Como no caso dos vetores podem ser utilizados comandos de repetição sobre tarefas que devem ser realizadas, de forma idêntica.
- No exemplo são utilizados dois comandos de repetição para ler os valores.
 - Um para cada dimensão da matriz.
- Cada posição da matriz é uma variável, a qual é identificada por dois índices.
 - O tempo para acessar qualquer uma das posições da matriz é o mesmo.

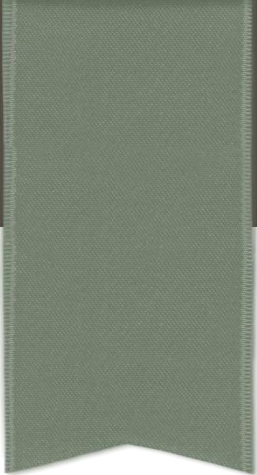
```
#include <stdio.h>

int main(void) {
    int mat[100][50];
    int i, j;
    for (i=0; i<100; i++)
        for (j=0; j<50; j++) {
            printf("Valor de mat[%d][%d]: ", i, j);
            scanf("%d", &mat[i][j]);
        }
    return 0;
}
```

Acessando um elemento do array bidimensional

- Qualquer posição da matriz pode aparecer em comandos como uma variável comum.

```
scanf("%d", &mat[5][0]);           //comando de leitura  
mat[0][0] = 10;                     //comando de atribuição  
mat[1][2] = mat[5][0] + mat[0][0]; //expressão
```



ARRAY COM MAIS DE DUAS DIMENSÕES

Array com mais de duas dimensões

- A linguagem C permite que se crie um array com mais de duas dimensões de maneira fácil.
- Cada dimensão do array é definida por um par de colchetes na sua declaração.

```
#include <stdio.h>

int main(void) {
    int vetor[15]; // Array com 1 dimensão
    float matriz[20][30]; // Array com 2 dimensões
    double cubo[10][10][10]; // Array com 3 dimensões
    char ch[5][5][5][5]; // Array com 4 dimensões
    return 0;
}
```

Array com mais de duas dimensões

- Exemplo: inicializando um array tridimensional com valores zero.

```
#include <stdio.h>

int main(void) {
    int cubo[10][10][10];
    int i, j, k;
    for (i=0; i<10; i++)
        for (j=0; j<10; j++)
            for (k=0; k<10; k++)
                cubo[i][j][k] = 0;
    return 0;
}
```

Array com mais de duas dimensões

- Apesar de terem o comportamento de estruturas com mais de uma dimensão, os dados dos arrays são sempre armazenados linearmente na memória.
 - Independentemente da quantidade de dimensões que possua.

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)

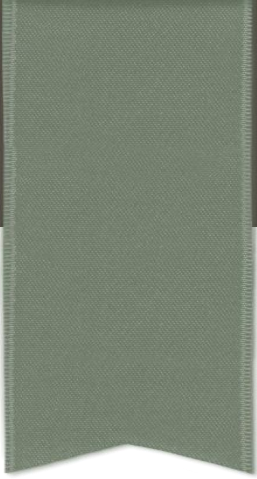
Array bidimensional

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(4,0)	(4,1)	(4,2)	(4,3)	(4,4)
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Disposição na memória

Array com mais de duas dimensões

- É importante ter em mente qual dimensão se move mais rapidamente na memória.
 - Sempre a mais à direita, independentemente do tipo ou da quantidade de dimensões do array.



INICIALIZAÇÃO DE ARRAYS

Inicialização de arrays

- A declaração de um array apenas reserva espaço na memória e não associa a ele nenhum valor.
- Seu valor inicial é o lixo de memória contido no espaço reservado.
- Dependendo do programa é necessário iniciar todas as posições do array com algum valor predefinido.

Inicialização de arrays

- O programa a seguir inicializa todas as posições dos arrays com o valor zero.

```
#include <stdio.h>
#define D1 10
#define D2 20
#define D3 30
int main(void) {
    int vetor[50], matriz[10][15], tri[D1][D2][D3];
    int i, j, k;
    for (i=0; i<50; i++)
        vetor[i] = 0;
    for (i=0; i<10; i++)
        for (j=0; j<15; j++)
            matriz[i][j] = 0;
    for (i=0; i<D1; i++)
        for (j=0; j<D2; j++)
            for (k=0; k<D3; k++)
                tri[i][j][k] = 0;
    return 0;
}
```

Inicialização de arrays

- Para colocar valores diferentes em cada posição é necessário fornecer valor a valor.
- Os dois programas são equivalentes.

```
#include <stdio.h>
```

```
int main(void) {  
    int vetor[5];  
    vetor[0] = 16;  
    vetor[1] = 22;  
    vetor[2] = 23;  
    vetor[3] = 1;  
    vetor[4] = 14;  
    return 0;  
}
```

```
#include <stdio.h>
```

```
int main(void) {  
    int vetor[5] = {16,22,23,1,14};  
    return 0;  
}
```

Inicialização de arrays

- A forma geral de inicialização de um array utilizando o operador de chaves é:

`tipo_dado nome_array[tam1][tam2]...[tamN] = { dados };`

- **dados** são uma lista de valores (do mesmo tipo do array) separados por vírgula e delimitados pelo operador de chaves.
- Os valores devem ser inseridos na mesma ordem em que serão colocados no array.
- A inicialização de um array utilizando o operador de chaves **só pode ser feita durante sua declaração.**

Inicialização de arrays

- Inicialização de um array com mais de uma dimensão.
- O programa gera como saída:

m1[0][0]= 1	m2[0][0]= 1	m3[0][0]= 1
m1[0][1]= 2	m2[0][1]= 2	m3[0][1]= 2
m1[0][2]= 3	m2[0][2]= 3	m3[0][2]= 3
m1[0][3]= 4	m2[0][3]= 4	m3[0][3]= 0
m1[1][0]= 5	m2[1][0]= 5	m3[1][0]= 4
m1[1][1]= 6	m2[1][1]= 6	m3[1][1]= 5
m1[1][2]= 7	m2[1][2]= 7	m3[1][2]= 0
m1[1][3]= 8	m2[1][3]= 8	m3[1][3]= 0
m1[2][0]= 9	m2[2][0]= 9	m3[2][0]= 6
m1[2][1]=10	m2[2][1]=10	m3[2][1]= 7
m1[2][2]=11	m2[2][2]=11	m3[2][2]= 8
m1[2][3]=12	m2[2][3]=12	m3[2][3]= 9

```
#include <stdio.h>

int main(void) {
    int m1[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};
    int m2[3][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};
    int m3[3][4] = {{1,2,3}, {4,5}, {6,7,8,9}};
    int i,j;
    for (i=0; i<3; i++)
        for (j=0; j<4; j++) {
            printf("m1[%d][%d]=%2d    ", i, j, m1[i][j]);
            printf("m2[%d][%d]=%2d    ", i, j, m2[i][j]);
            printf("m3[%d][%d]=%2d\n", i, j, m3[i][j]);
        }
    return 0;
}
```

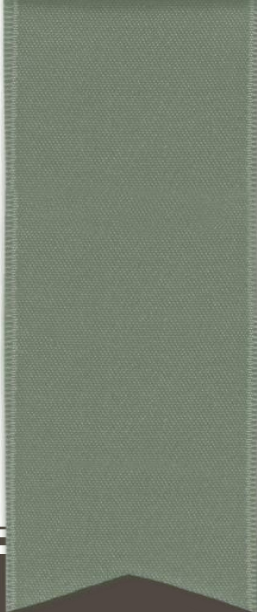
Inicialização de arrays

- A inicialização de um array e caracteres pode ser da forma vista anteriormente, mas existe forma mais direta.

```
#include <stdio.h>

int main(void) {
    char str1[12] = {'L','i','n','g','u','a','g','e','m',' ',' ','C','\0'};
    char str2[12] = "Linguagem C";
    char str3[2][12] = {"Linguagem C", "Muito boa"};
    return 0;
}
```

- Na inicialização de um array de caracteres sempre é acrescentado um caractere extra no final, cujo valor é `'\0'`.



INICIALIZAÇÃO SEM TAMANHO

Inicialização sem tamanho

- A linguagem C permite inicializar um array sem que seu tamanho seja definido.
 - Os **colchetes** aparecem **sem conteúdo**.
- O compilador contará quantos valores existem na lista de dados e considerará essa quantidade como o tamanho do array.
- Isso **ocorre na etapa de compilação do programa**.
 - Depois disso **o tamanho do array não poderá ser modificado**.

```
#include <stdio.h>

int main(void) {
    // Array de inteiros com 5 posições
    int vetor[] = {1,2,3,4,5};
    return 0;
}
```

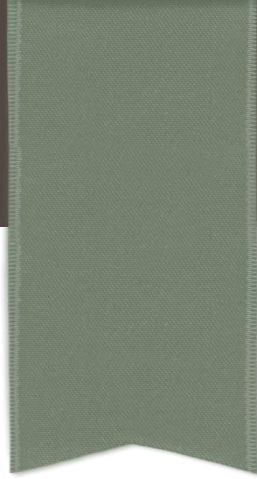
Inicialização sem tamanho

- A inicialização sem tamanho é muito útil quando não queremos contar quantos caracteres serão necessários para inicializar um array de caracteres.

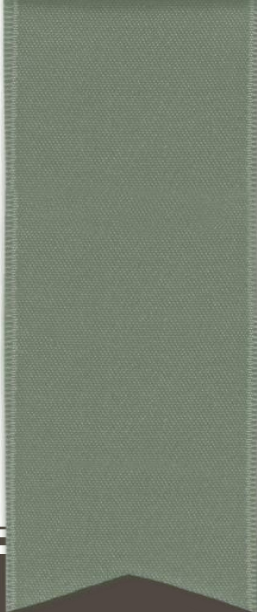
```
#include <stdio.h>

int main(void) {
    char str[] = "Meu array";
    return 0;
}
```

- Qual o tamanho do array criado no exemplo?
 - Resposta: 10 (9 caracteres imprimíveis + '\0').



EXEMPLOS

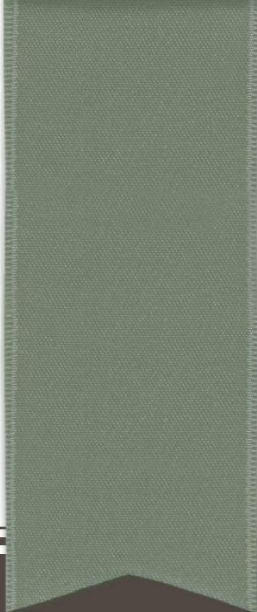


SOMA DOS ELEMENTOS DE UM VETOR

Soma dos elementos de um vetor

```
#include <stdio.h>

int main(void) {
    int v[7] = {16,22,23,1,14,29,18};
    int soma, i;
    soma = 0;
    for (i=0; i<7; i++)
        soma = soma + v[i];
    printf("Soma: %d\n", soma);
    return 0;
}
```

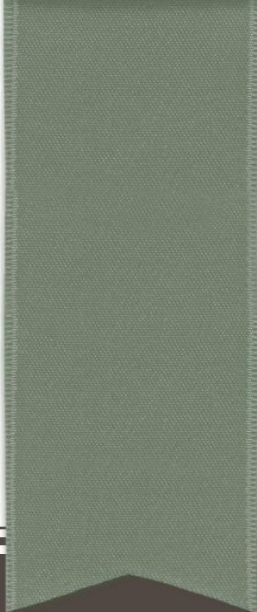


MAIOR VALOR DE UM VETOR

Maior valor de um vetor

```
#include <stdio.h>

int main(void) {
    int v[7] = {16,22,23,1,14,29,18};
    int maior, i;
    maior = v[0];
    for (i=1; i<7; i++)
        if (maior < v[i])
            maior = v[i];
    printf("Maior: %d\n", maior);
    return 0;
}
```

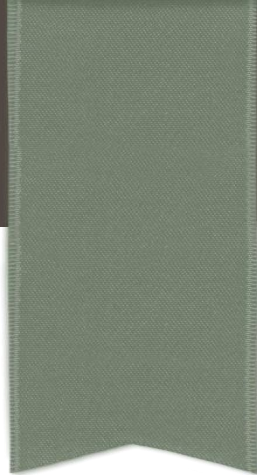


SOMA DOS ELEMENTOS DE UMA MATRIZ

Soma dos elementos de uma matriz

```
#include <stdio.h>

int main(void) {
    int m[3][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};
    int i, j, soma;
    soma = 0;
    for (i=0; i<3; i++)
        for (j=0; j<4; j++)
            soma = soma + m[i][j];
    printf("Soma: %d\n", soma);
    return 0;
}
```



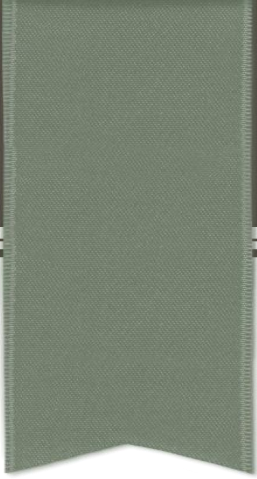
EXERCÍCIOS

Exercícios – vetores

1. Faça um programa que leia 6 valores inteiros e em seguida mostre na tela os valores na ordem em que foram lidos e na ordem inversa.
2. Faça um programa que leia 5 valores reais e em seguida mostre o menor, o maior e a média.
3. Faça um programa que leia um vetor de 8 posições. Em seguida leia 2 valores X e Y quaisquer correspondentes a duas posições no vetor. Exibir a soma dos valores encontrados nas posições X e Y.
4. Faça um programa que preencha um vetor com 5 números reais. Em seguida mostre na tela a quantidade de números negativos e a soma dos números positivos desse vetor.
5. Faça um programa que leia 2 arrays A e B de 4 posições cada. Em seguida calcule e mostre o conteúdo do array $C = A - B$.
6. Faça um programa que preencha um vetor com 10 números inteiros. O programa não deverá permitir que sejam inseridos no vetor 2 números de igual valor.

Exercícios – matrizes

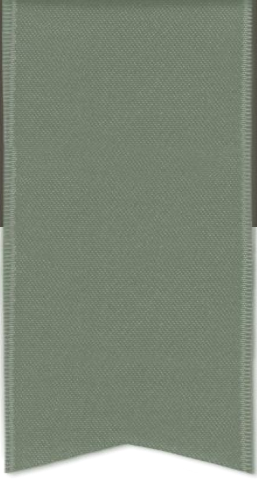
1. Faça um programa que leia uma matriz 4x4 e mostre o maior valor contido nessa matriz e a sua localização (linha e coluna).
2. Faça um programa que declare uma matriz 5x5, preencha com 1 a diagonal principal e com 0 os demais elementos. Ao final mostre a matriz.
3. Leia uma matriz de tamanho 3x3 e em seguida imprima a soma dos valores contidos em sua diagonal secundária.
4. Calcular e imprimir na tela uma matriz de tamanho 10x10, em que seus elementos são da forma:
 $A[i][j] = \{ \begin{matrix} 3i+j, & \text{se } i < j \\ 2i^2-2, & \text{se } i=j \\ 2j-5i & \text{se } i > j \end{matrix} \}$.
5. Faça um programa que leia uma matriz 3x3 e calcule a matriz inversa.
6. Faça um programa que permita ao usuário entrar com uma matriz de tamanho 3x3 de números inteiros. Em seguida, calcule um vetor de 3 posições, em que cada posição deverá armazenar a soma dos números de cada coluna da matriz.
7. Faça um programa que leia uma matriz de tamanho 5x3 com as notas de 5 alunos em três provas. Mostre na tela a média de cada aluno e quantos obtiveram média maior ou igual a 6.
8. Faça um programa que leia 2 matrizes 2x3 e calcule a matriz soma. Mostre a matriz soma.



ARRAYS DE CARACTERES (STRINGS)

```
char str[] = "String";
```

índice →	0	1	2	3	4	5	6
str →	S	t	r	i	n	g	\0



DEFINIÇÃO E DECLARAÇÃO DE STRINGS

Definição e declaração de strings

- **String** é o nome usado para definir uma sequência de caracteres adjacentes na memória do computador.
 - Pode ser uma palavra ou frase.
 - É armazenada na memória do computador na forma de um array do tipo char.
- Como é um array de caracteres, sua declaração segue as mesmas regras da declaração de um array convencional:
- A declaração **char str[6]** cria na memória do computador um array de caracteres (string) de nome **str** e tamanho igual a **6**.
- Apesar de ser um array as strings têm no elemento seguinte a última letra da palavra/frase armazenada, um caractere **'\0'**.

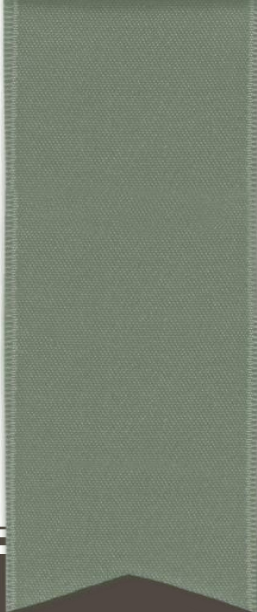
Definição e declaração de strings

- O caractere `'\0'` indica o fim da sequência de caracteres.

```
char str[20] = "Seja bem-vindo.";
```

S	e	j	a		b	e	m	-	v	i	n	d	o	.	\0	#	K	ç	~
---	---	---	---	--	---	---	---	---	---	---	---	---	---	---	----	---	---	---	---

- As posições restantes da string (após o `'\0'`) não estão sendo utilizadas no momento.
- Ao definir o tamanho de uma string deve-se considerar o caractere `'\0'`.
- Como o caractere `'\0'` indica o final da string, um array de caracteres com N posições pode armazenar, no máximo, uma string com até N-1 caracteres.



INICIALIZANDO UMA STRING

Iniciando uma string

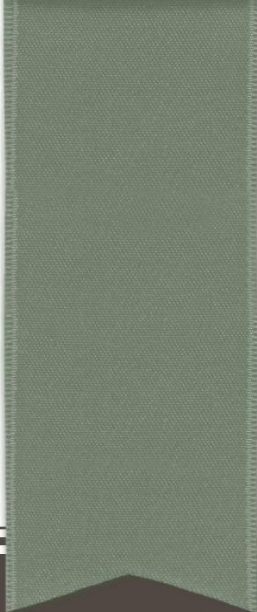
- Uma string pode ser lida ou já ser definida com um valor inicial.
- Pode-se inicializá-la do mesmo que vetores e matrizes:

```
char str1[20] = {'L','i','n','g','u','a','g','e','m',' ',' ','C','\0'};
```

- Essa forma de inicialização não é muito prática.
- A inicialização de strings também pode ser feita com aspas duplas:

```
char str2[20] = "Linguagem C";
```

- Essa forma de inicialização possui a vantagem de já inserir o caractere ‘\0’ no final da string.



ACESSANDO UM ELEMENTO DA STRING

Acessando um elemento da string

- Como se trata de um array, cada caractere pode ser acessado individualmente como em qualquer outro array.

```
char str[6] = "Cinto";
```

C	i	n	t	o	\0
---	---	---	---	---	----

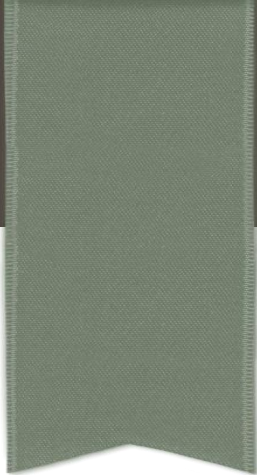
```
str[0] = 'T';
```

T	i	n	t	o	\0
---	---	---	---	---	----

```
str[4] = 'a';
```

T	i	n	t	a	\0
---	---	---	---	---	----

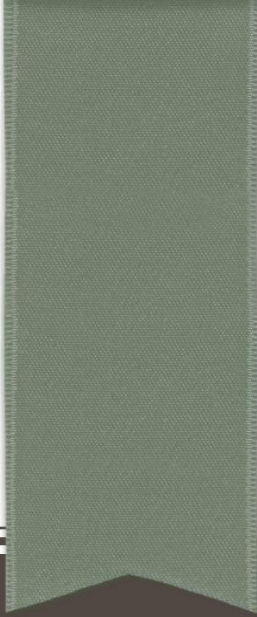
- Na atribuição de strings, usam-se “aspas duplas”, enquanto na de caracteres usam-se ‘aspas simples’.



TRABALHANDO COM STRINGS

Trabalhando com strings

- O primeiro cuidado com strings é na **operação de atribuição**.
- Uma string é um array, e a linguagem C não suporta a atribuição de um array para outro.
- Para atribuir o conteúdo de uma string a outra, **o correto é copiar a string, elemento por elemento para a outra string**.
- A biblioteca padrão da linguagem C possui funções especialmente desenvolvidas para a manipulação de strings.
 - Biblioteca **<string.h>**.



LENDO UMA STRING DO TECLADO

Lendo uma string do teclado

- Existem várias maneiras de se fazer a leitura de uma sequência de caracteres do teclado.
- Uma delas é utilizando a função `scanf()` com o formato de dados “%s”:

```
char str[20];  
scanf("%s", str);
```

- Quando usamos a função `scanf()` para ler uma string, o símbolo & antes do nome da variável não é utilizado.
- Os colchetes também não são utilizados.

Lendo uma string do teclado

- Infelizmente, para muitos casos, a função `scanf()` não é a melhor opção para se ler uma string do teclado.
- A função **`scanf()`** lê apenas strings digitadas sem espaços.
 - Lê somente as palavras.
- Se for digitada uma frase (uma sequência de caracteres contendo espaços), **apenas os caracteres digitados antes do primeiro espaço encontrado serão armazenados na string** se a sua leitura for feita com a função **`scanf()`**.

Lendo uma string do teclado

```
#include <stdio.h>

int main(void) {
    char str[30];
    printf("Digite uma palavra: ");
    scanf("%s", str);
    printf("A palavra digitada foi:\n%s\n", str);
    return 0;
}
```

Lendo uma string do teclado

- Uma alternativa mais eficiente para a leitura de uma string é a função `gets()`.
- Faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços) até encontrar um “enter”.

```
char str[20];  
gets(str);
```

```
#include <stdio.h>  
  
int main(void) {  
    char str[30];  
    printf("Digite uma frase: ");  
    gets(str);  
    printf("A frase digitada foi:\n%s\n", str);  
    return 0;  
}
```

Lendo uma string do teclado

- Existe outra função que, utilizada de forma adequada, também permite a leitura de strings do teclado.
- É a função **fgets()**, cujo protótipo é:

```
char *fgets(char *str, int tamanho, FILE *fp);
```

- A função **fgets()** recebe três parâmetros de entrada:
 - **str** → A variável que armazenará a string lida.
 - **tamanho** → O limite máximo de caracteres a serem lidos.
 - **fp** → Variável que está associada ao arquivo de onde a string será lida. Utilizar **stdin** para ler do teclado.
- E retorna:
 - **NULL** → No caso de erro ou fim do arquivo.
 - O ponteiro para o primeiro caractere da string recuperada em **str**.

Lendo uma string do teclado

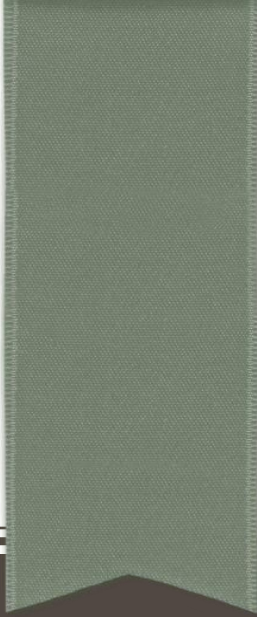
- No exemplo serão lidos no máximo 30 caracteres para a variável **str**, incluindo o '\0';

```
#include <stdio.h>

int main(void) {
    char str[30];
    printf("Digite uma frase: ");
    fgets(str, 30, stdin);
    printf("A frase digitada foi:\n%s\n", str);
    return 0;
}
```

Lendo uma string do teclado

- A função **fgets()** possui algumas diferenças e vantagens sobre a **gets()**.
- Na função **fgets()**, o caractere de nova linha (\n) fará parte da string, o que não acontece com a **gets()**.
- A função **fgets()** especifica o tamanho máximo da string de entrada.
- Diferentemente da função **gets()**, a função **fgets()** lê a string até que um caractere de nova linha seja lido ou “tamanho-1” caracteres tenham sido lidos.
 - **Evita o estouro do buffer**, que ocorre quando se tenta ler algo maior do que pode ser armazenado na string.



LIMPANDO O BUFFER DO TECLADO

Limpendo o buffer do teclado

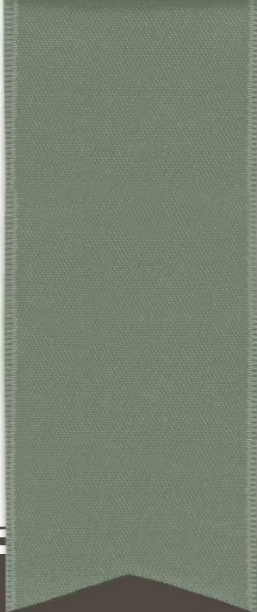
- Podem ocorrer erros durante a leitura de caracteres ou strings do teclado.
- Para limpar o buffer do teclado (entrada-padrão) utiliza-se a função `setbuf(stdin, NULL)` antes de realizar a leitura de caracteres ou strings.
- A função `setbuf()` preenche um buffer (primeiro parâmetro) com determinado valor (segundo parâmetro).
- No exemplo o buffer da entrada-padrão (stdin - o teclado) é preenchido com o valor vazio (NULL).
- Um buffer preenchido com NULL é considerado limpo/vazio.

Limpendo o buffer do teclado

```
#include <stdio.h>

int main(void) {
    char palavra[30];
    float valor;
    printf("Digite uma palavra: ");
    scanf("%s", palavra);
    printf("Digite um valor: ");
    setbuf(stdin, NULL);
    scanf("%f", &valor);
    printf("A palavra digitada foi: %s\n", palavra);
    printf("O valor digitado foi: %f\n", valor);
    return 0;
}
```





ESCREVENDO UMA STRING NA TELA

Escrevendo uma string na tela

- Para escrever uma string na tela pode ser utilizada a função `printf()` com o formato de dados `“%s”`.
- Os colchetes não são utilizados.

```
#include <stdio.h>

int main(void) {
    char str[] = "Ola mundo!!!";
    printf("%s\n", str);
    return 0;
}
```

Escrevendo uma string na tela

- Outra função que pode ser utilizada é a `fputs()`, cujo protótipo é:

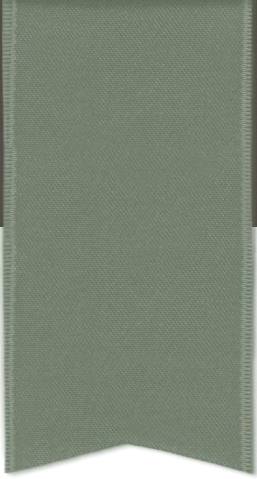
```
int fputs(char *str, FILE *fp) ;
```

- A função **`fputs()`** recebe dois parâmetros de entrada:
 - `str` → A string (array de caracteres) a ser escrita na tela.
 - `fp` → A variável que está associada ao arquivo onde a string será escrita. Utilizar **`stdout`** para escrever na tela.
- E retorna:
 - A constante EOF (em geral -1), se houver erro na escrita.
 - Um valor diferente de ZERO se o texto for escrito com sucesso.

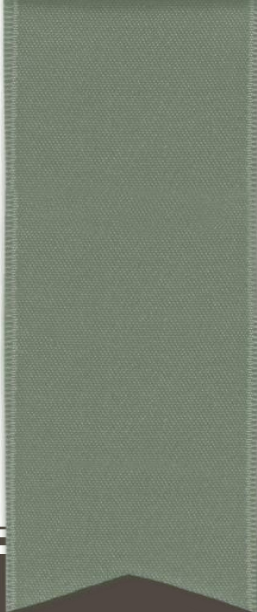
Escrevendo uma string na tela

```
#include <stdio.h>

int main(void) {
    char str[] = "Ola mundo!!!";
    fputs(str, stdout);
    return 0;
}
```



FUNÇÕES PARA MANIPULAÇÃO DE STRINGS



TAMANHO DE UMA STRING

Tamanho de uma string

- Para obter o tamanho de uma string usa-se a função **strlen()**, cujo protótipo é:

```
int strlen(char *str);
```

- A função **strlen()** recebe como parâmetro uma string e retorna a quantidade de caracteres armazenados na string.

Tamanho de uma string



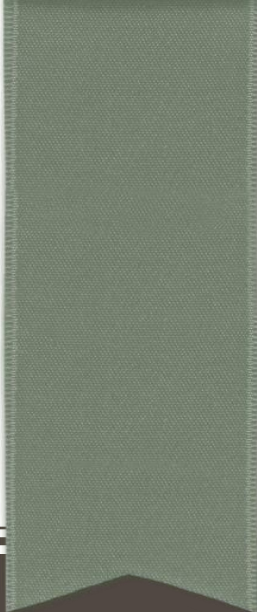
```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str[30] = "Mar azul";
    int tamanho;
    tamanho = strlen(str);
    printf("A string possui %d caracteres.\n", tamanho);
    return 0;
}
```

- O programa apresenta como saída:

A string possui 8 caracteres.

- 8 é a quantidade de caracteres antes do '\0', não o tamanho do array de caracteres.



COPIANDO UMA STRING

Copiando uma string

- A única forma de atribuir o conteúdo de uma string a outra é a cópia, elemento por elemento, de uma string para outra.
- A função **strcpy()** realiza essa tarefa.

char* strcpy(char *destino, char *origem);

- A função **strcpy()** copia a sequência de caracteres contida em **origem** para o array de caracteres **destino** e retorna o endereço da string destino.
- Para evitar estouro de buffer, **o tamanho do array destino deve ser longo o suficiente para conter a sequência de caracteres contida em origem.**

Copiando uma string

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char origem[30] = "Ola mundo!!!";
    char destino[30];
    strcpy(destino, origem);
    printf("%s\n", destino);
    return 0;
}
```

Copiando uma string

- Para limitar a quantidade de caracteres a serem copiados pode ser utilizada a função **strncpy()**, que possui protótipo:

```
char* strncpy(char *destino, char *origem, size_t num);
```

- A função **strncpy()** copia os primeiros **num** caracteres contidos em **origem** para o array de caracteres **destino** e retorna o endereço da string destino.

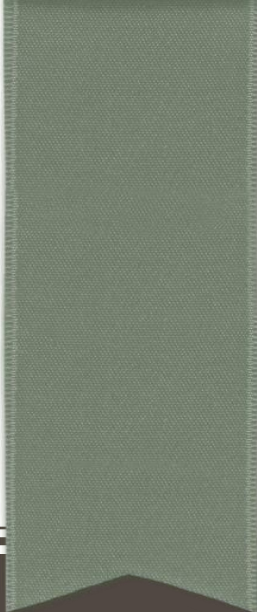
Copiando uma string

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char origem[30] = "abcdefgh";
    char destino[30];
    strncpy(destino, origem, 5);
    printf("%s\n", destino);
    return 0;
}
```

- O programa apresenta como saída:

abcde



COMPARANDO DUAS STRINGS

Comparando duas strings

- Da mesma maneira como o operador de atribuição não funciona para strings, o mesmo ocorre com operadores relacionais.
- Para saber se duas strings são iguais usa-se a função `strcmp()`:

```
int strcmp(char *str1, char *str2);
```

- A função `strcmp()` compara posição a posição as duas strings (`str1` e `str2`) e retorna um valor inteiro:
 - Igual a zero, no caso de as duas strings serem iguais.
 - Menor que zero se `str1` for alfabeticamente menor que `str2`.
 - Maior que zero se `str1` for alfabeticamente maior que `str2`.
- A função `strcmp()` é case-sensitive.
 - Diferencia letras maiúsculas e minúsculas.

Comparando duas strings

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str1[50], str2[50];
    printf("Entre com a string 1: ");
    gets(str1);
    printf("Entre com a string 2: ");
    gets(str2);
    if (!strcmp(str1, str2))
        printf("As strings sao iguais.\n");
    else
        printf("As strings sao diferentes.\n");
    return 0;
}
```

Comparando duas strings

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char str1[50], str2[50];
    int cmp;
    printf("Entre com a string 1: ");
    gets(str1);
    printf("Entre com a string 2: ");
    gets(str2);
    cmp = strcmp(str1, str2);
    if (cmp == 0)
        printf("As strings sao iguais.\n");
    else if (cmp < 0)
        printf("%s vem antes de %s.\n", str1, str2);
    else
        printf("%s vem antes de %s.\n", str2, str1);
    return 0;
}
```

Comparando duas strings

- Outra função para comparação de strings é a `strncmp()`, que possui protótipo:

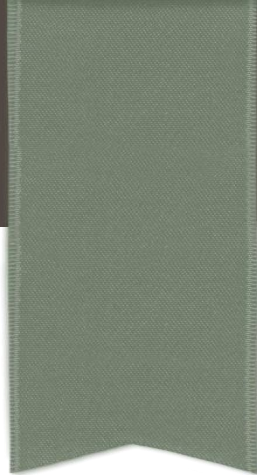
```
int strncmp(char *str1, char *str2, size_t num);
```

- `strncmp()` é semelhante à `strcmp()`, sendo que compara somente os `num` primeiros caracteres de `str1` e `str2`.

Comparando duas strings

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str1[50], str2[50];
    printf("Entre com a string 1: ");
    gets(str1);
    printf("Entre com a string 2: ");
    gets(str2);
    if (!strncmp(str1, str2, 4))
        printf("Os 4 primeiros caracteres das strings sao iguais.\n");
    else
        printf("Existe diferenca entre os 4 primeiros caracteres.\n");
    return 0;
}
```



EXERCÍCIOS

Exercícios

1. Faça um programa que leia uma string, um número N e imprima as N primeiras letras da string.
2. Sem usar a função **strlen()**, faça um programa que leia uma string e imprima quantos caracteres ela possui.
3. Faça um programa que leia uma string e conte quantas vogais ela possui.
4. Faça um programa que leia uma string e a coloque invertida na mesma variável.
5. Faça um programa que leia uma string e verifique se ela é um palíndromo. Um palíndromo é uma palavra ou frase que pode ser lida de trás para frente que mantém o conteúdo. Exemplos: “osso”, “radar” e “sopapos”.
6. Faça um programa que leia uma string e a imprima em letras maiúsculas. Dica: subtraia 32 dos caracteres entre 97 e 122.
7. O código de César é uma das técnicas de criptografia mais simples e conhecidas. É um tipo de substituição no qual cada letra do texto é substituída por outra, que se apresenta n posições após ela no alfabeto. Por exemplo, com uma troca de três posições, a letra A seria substituída por D, B se tornaria E e assim por diante. Escreva um programa que faça uso desse código de César para três posições. Entre com uma string e imprima a string codificada.

TIPOS DEFINIDOS PELO PROGRAMADOR

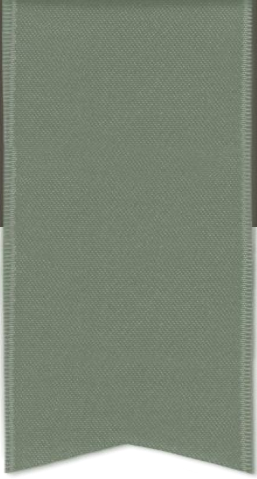
```
typedef union
{
    struct
    {
        float x;
        float y;
        float z;
        float w;
    };
    float OpenGL[4];
}Vector;
```

MEMORY

	[0]	0x100
	[1]	0x104
	[2]	0x108
	[3]	0x10b
		0x10f

Tipos definidos pelo programador

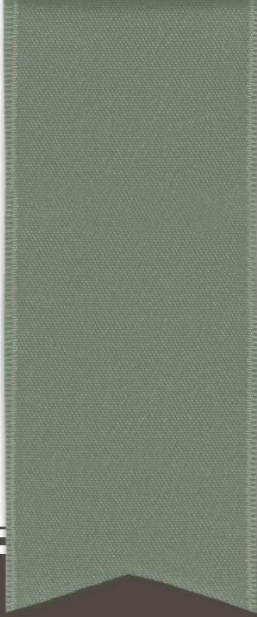
- Os tipos de variáveis vistos até agora podem ser classificados em duas categorias:
 - Tipos básicos: char, int, float, double e void.
 - Tipos compostos homogêneos: array.
- Dependendo da situação esses tipos podem não ser suficientes.
- A linguagem C permite criar novos tipos de dados a partir dos tipos básicos. Estes tipos são:
 - **Estruturas**: comando **struct**.
 - **Unões**: comando **union**.
 - **Enumerações**: comando **enum**.
 - **Nomeação** um tipo existente: comando **typedef**.



ESTRUTURAS: STRUCT

Estruturas: struct

- Uma estrutura pode ser vista como um conjunto de variáveis sob o mesmo nome.
- Cada uma delas pode ter qualquer tipo.
- A ideia é criar apenas um tipo de dado que contenha vários membros.
 - Cada membro nada mais é do que outra variável.



DECLARAÇÃO DE UMA ESTRUTURA (STRUCT)

Declaração de uma estrutura (struct)

- A forma geral da definição de uma nova estrutura é:

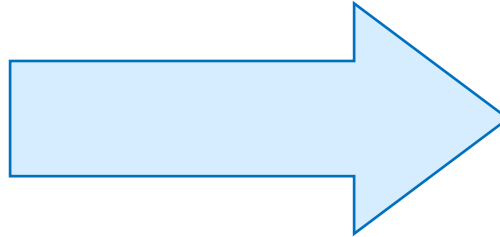
```
struct nome_struct {  
    tipo1 campo1;  
    tipo2 campo2;  
    ...  
    tipoN campoN;  
};
```

- Uma estrutura permite agrupar de forma organizada vários tipos de dados diferentes em um único local.

Declaração de uma estrutura (struct)

Pessoa

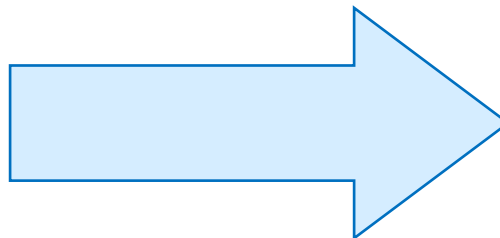
```
char nome[40];  
char endereco[50];  
float salario;  
int dependentes;
```



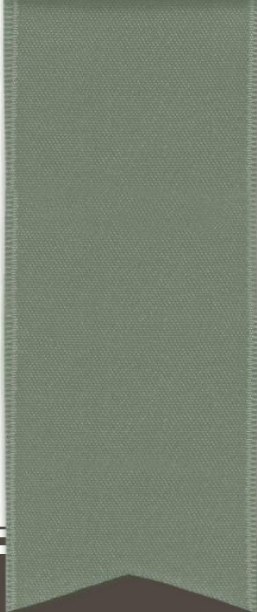
```
struct pessoa {  
    char nome[40];  
    char endereco[50];  
    float salario;  
    int dependentes;  
};
```

Aluno

```
char nome[40];  
int matricula;  
float nota1;  
float nota2;  
float nota3;
```



```
struct aluno {  
    char nome[40];  
    int matricula;  
    float nota1, nota2, nota3;  
};
```



DECLARANDO UMA VARIÁVEL DO TIPO DA ESTRUTURA (STRUCT)

Declarando uma variável do tipo da estrutura (struct)

- Definida a estrutura, uma variável pode ser declarada normalmente:

```
struct pessoa p;
```

- Por ser um tipo definido pelo programador, usa-se a palavra **struct** antes do tipo da variável declarada.
- O uso de estruturas facilita a manipulação de dados do programa.
- Imagine declarar 5 cadastros para 5 pessoas diferentes:

```
char nome1[40], nome2[40], nome3[40], nome4[40], nome5[40];  
char endereco1[50], endereco2[50], endereco3[50], endereco4[50], endereco5[50];  
float salario1, salario2, salario3, salario4, salario5;  
int dependentes1, dependentes2, dependentes3, dependentes4, dependentes5;
```

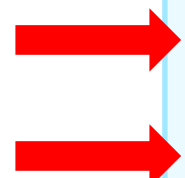
- Com estruturas a declaração seria:

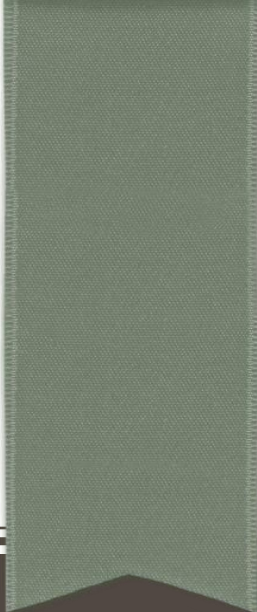
```
struct pessoa p1, p2, p3, p4, p5;
```

Declarando uma variável do tipo da estrutura (struct)

- As declarações das variáveis podem ser feitas logo após a declaração da estrutura (**p1** e **p2** no exemplo) ou normalmente utilizando a declaração **struct nome_estrutura** (**p3** e **p4** no exemplo).

```
struct pessoa {  
    char nome[40];  
    char endereco[50];  
    float salario;  
    int dependentes;  
} p1, p2;  
  
struct pessoa p3, p4;
```





ACESSANDO OS CAMPOS DE UMA ESTRUTURA (STRUCT)

Acessando os campos de uma estrutura (struct)

- Os campos de uma variável do tipo estrutura podem ser acessados usando o operador “.” (ponto).

```
#include <stdio.h>
#include <string.h>

struct pessoa {
    char nome[40];
    char endereco[50];
    float salario;
    int dependentes;
};
```

```
int main(void) {
    struct pessoa p;
    strcpy(p.nome, "Pedro Alvares Cabral");
    strcpy(p.endereco, "Rua sobe e desce, nr 0");
    p.salario = 1500.04;
    p.dependentes = 7;
    printf("Nome: %s\nEndereco: %s\nSalario: %.2f\nDependentes: %d\n", p.nome, p.endereco, p.salario, p.dependentes);
    return 0;
}
```

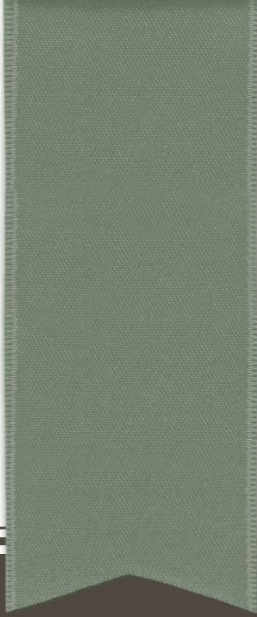
Acessando os campos de uma estrutura (struct)

- O mesmo programa, mas lendo os dados.

```
#include <stdio.h>
```

```
struct pessoa {  
    char nome[40];  
    char endereco[50];  
    float salario;  
    int dependentes;  
};
```

```
int main(void) {  
    struct pessoa p;  
    printf("Nome: ");  
    gets(p.nome);  
    printf("Endereco: ");  
    gets(p.endereco);  
    printf("Salario: ");  
    scanf("%f", &p.salario);  
    printf("Dependentes: ");  
    scanf("%d", &p.dependentes);  
    printf("\nNome: %s\nEndereco: %s\nSalario:  
%.2f\nDependentes: %d\n", p.nome, p.endereco, p.salario,  
p.dependentes);  
    return 0;  
}
```



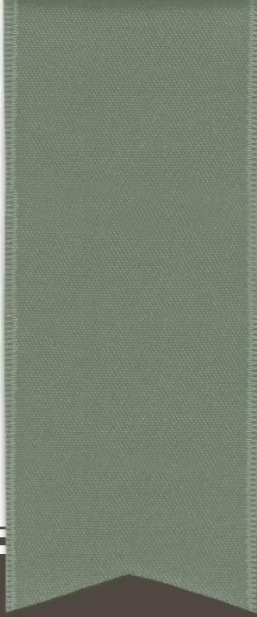
INICIALIZAÇÃO DE ESTRUTURAS (STRUCT)

Inicialização de estruturas (struct)

- Uma estrutura pode ser inicializada independentemente do tipo das variáveis contidas nela.
- Na declaração da variável basta definir uma lista de valores separados por vírgula e delimitados pelo operador de chaves.

```
struct pessoa p = { "Pedro Alvares", "Rua Sobe e Desce", 1500.04, 7};
```

- Elementos omitidos são inicializados com 0.
 - Se for uma string, será inicializada com uma string vazia (“”).



ARRAY DE ESTRUTURAS

Array de estruturas

- Voltemos ao problema do cadastro de pessoas.
- O uso de estruturas facilita muito a manipulação dos dados do programa.
- Vimos que pessoas podem ser facilmente cadastradas se declaradas como variáveis de estrutura.

```
struct pessoa p1, p2, p3, p4, p5;
```

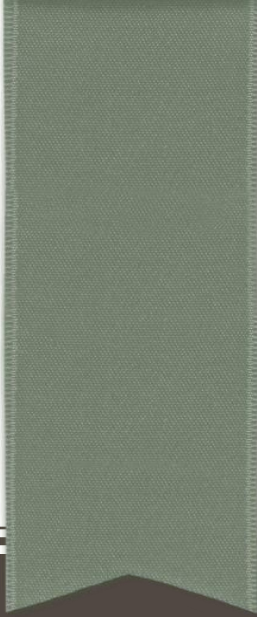
- A representação desses cadastros pode ser ainda mais simplificada, se utilizarmos o conceito de arrays:

```
struct pessoa p[5];
```

- Em um array de estruturas, o operador de ponto vem depois dos colchetes do índice do array.

```
p[3].dependentes = 3;
```

```
#include <stdio.h>
struct pessoa {
    char nome[40];
    char endereco[50];
    float salario;
    int dependentes;
};
int main(void) {
    struct pessoa p[5];
    int i;
    for (i=0; i<5; i++) {
        setbuf(stdin, NULL);
        printf("\nNome[%d]: ", i);
        gets(p[i].nome);
        printf("Endereco[%d]: ", i);
        gets(p[i].endereco);
        printf("Salario[%d]: ", i);
        scanf("%f", &p[i].salario);
        printf("Dependentes[%d]: ", i);
        scanf("%d", &p[i].dependentes);
    }
    for (i=0; i<5; i++)
        printf("\nNome: %s\nEndereco: %s\nSalario: %.2f\nDependentes: %d\n", p[i].nome,
p[i].endereco, p[i].salario, p[i].dependentes);
    return 0;
}
```



ATRIBUIÇÃO ENTRE ESTRUTURAS (STRUCTS)

Atribuição entre estruturas (structs)

- As únicas operações possíveis em uma estrutura são:
 - Acesso aos membros da estrutura, por meio do operador ponto (.).
 - Atribuição, por meio do operador igual (=).
- Atribuições entre estruturas **só podem ser feitas quando as estruturas possuem o mesmo nome.**
 - Mesmo que possuam a mesma estrutura a atribuição não será permitida se não tiverem o mesmo nome.

```
struct pessoa p1, p2;
```

```
...
```

```
p2 = p1;
```

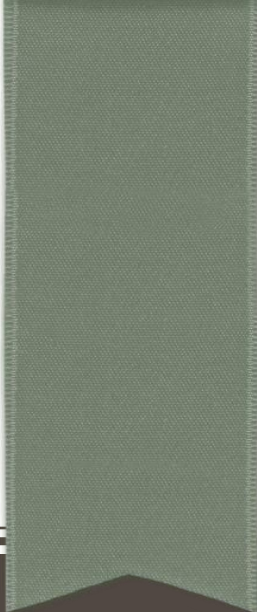
- No exemplo o conteúdo de p1 é copiado para p2.

Atribuição entre estruturas (structs)

```
#include <stdio.h>

struct pessoa {
    char nome[40];
    char endereco[50];
    float salario;
    int dependentes;
};

int main(void) {
    struct pessoa p1 = { "Pedro Alvares", "Rua Sobe e Desce", 1500.04, 7};
    struct pessoa p2;
    p2 = p1;
    printf("\nNome: %s\nEndereco: %s\nSalario: %.2f\nDependentes: %d\n",
p2.nome, p2.endereco, p2.salario, p2.dependentes);
    return 0;
}
```



ESTRUTURAS ANINHADAS

Estruturas aninhadas

- Uma estrutura pode agrupar uma quantidade arbitrária de campos de tipos diferentes.
- Uma estrutura também é um tipo de dado.
- Assim é possível declarar uma estrutura que possua um campo do tipo de outra estrutura previamente definida.

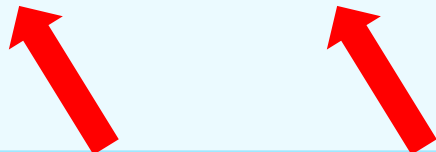
Estruturas aninhadas

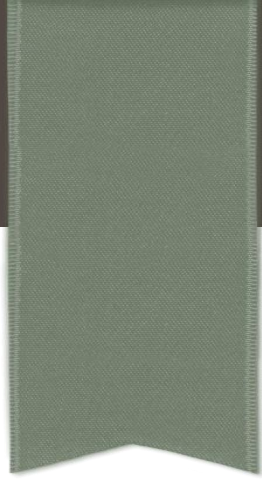
```
#include <stdio.h>

struct endereco {
    char rua[50];
    int numero;
};

struct pessoa {
    char nome[40];
    struct endereco end;
    float salario;
    int dependentes;
};

int main(void) {
    struct pessoa p = { "Pedro Alvares", {"Rua Sobe e Desce", 18}, 1500.04, 7};
    printf("\nNome: %s\nRua: %s\nNumero: %d\nSalario: %.2f\nDependentes: %d\n",
p.nome, p.end.rua, p.end.numero, p.salario, p.dependentes);
    return 0;
}
```

Two red arrows originate from the bottom of the slide. One arrow points to the expression `p.end.rua` in the `printf` statement, and the other points to the expression `p.end.numero`. These arrows highlight the use of the nested `end` struct within the `pessoa` struct.



UNIÕES: UNION

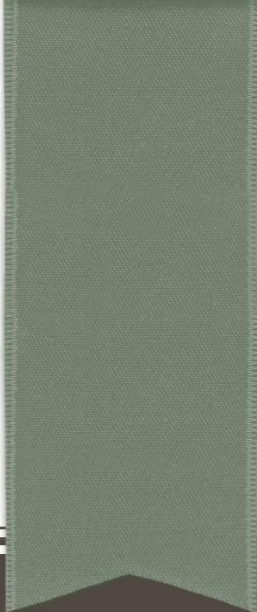
Unões: union

- Uma união pode ser vista como uma coleção de variáveis, onde cada uma delas pode ter qualquer tipo, mas somente uma pode ser utilizada por vez.
- A ideia da união é similar à da estrutura: criar apenas um tipo de dado que contenha vários membros, que nada mais são do que outras variáveis.
- Tanto a declaração quanto o acesso aos elementos de uma união são similares aos de uma estrutura.

Unões: union

- A forma geral da definição de união utiliza o comando **union**:

```
union nome_union {  
    tipo1 campo1;  
    tipo2 campo2;  
    ...  
    tipoN campoN;  
};
```



DIFERENÇA ENTRE ESTRUTURA E UNIÃO

Diferença entre estrutura e união

- Todos os elementos contidos na união ocupam o mesmo espaço físico na memória.
- Uma estrutura (**struct**) reserva espaço de memória para todos os seus elementos.
- Uma união (**union**) reserva espaço de memória para o seu maior elemento e compartilha essa memória com os demais.

Diferença entre estrutura e união

- Exemplo:

```
union tipo {  
    char ch;  
    int val;  
};
```

- Essa união possui o nome **tipo** e duas variáveis:
 - ch do tipo char (1 byte).
 - val do tipo int (4 bytes).
- Uma variável declarada desse tipo ocupará 4 bytes na memória, que é o tamanho do maior dos elementos da união.
- Em uma união apenas um membro pode ser armazenado de cada vez.
 - Cabe ao programador saber qual dado foi mais recentemente armazenado em uma união.

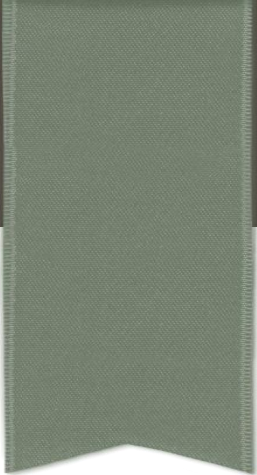
Diferença entre estrutura e união

```
#include <stdio.h>
union tipo {
    char ch;
    int i;
    float f;
};
int main(void) {
    union tipo x;
    printf("ch=%c  int=%d  float=%f\n", x.ch, x.i, x.f);
    x.ch = 'A';
    printf("ch=%c  int=%d  float=%f\n", x.ch, x.i, x.f);
    x.i = 1234567890;
    printf("ch=%c  int=%d  float=%f\n", x.ch, x.i, x.f);
    x.f = 1234567890;
    printf("ch=%c  int=%d  float=%f\n", x.ch, x.i, x.f);
    return 0;
}
```

Diferença entre estrutura e união

- O programa anterior apresenta como saída:

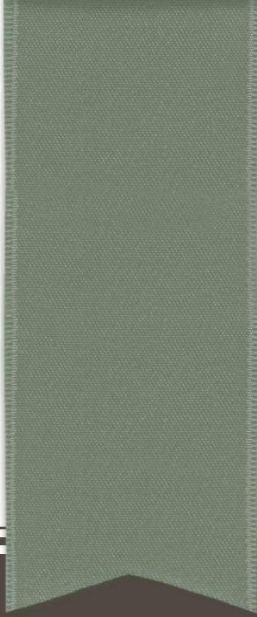
```
ch=é   int=130   float=0.000000
ch=A   int=65    float=0.000000
ch=Ê   int=1234567890   float=1228890.250000
ch=♠   int=1318267910   float=1234567936.000000
```



ENUMERAÇÕES: ENUM

Enumerações: enum

- Uma enumeração pode ser vista como uma lista de constantes, em que cada constante possui um nome significativo.
- A ideia é criar apenas um tipo de dado que contenha várias constantes.
- Uma variável desse tipo só poderá receber como valor uma dessas constantes.



DECLARANDO UMA ENUMERAÇÃO (ENUM)

Declarando uma enumeração (enum)

- A forma geral de uma enumeração utiliza o comando **enum**:

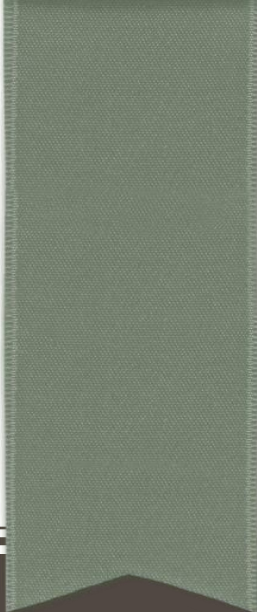
```
enum nome_enum { lista_de_identificadores };
```

- **lista_de_identificadores** é uma lista de palavras separadas por vírgula e delimitadas pelo operador de chaves.
 - Essas palavras constituem as constantes definidas pela enumeração.

- Exemplo:

```
enum semana {Domingo, Segunda, Terca, Quarta, Quinta, Sexta, Sabado};
```

- cria uma enumeração de nome **semana**, cujos valores constantes são os nomes dos dias da semana.



DECLARANDO UMA VARIÁVEL DO TIPO DA ENUMERAÇÃO (ENUM)

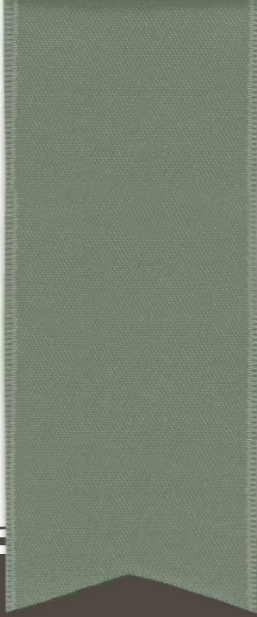
Declarando uma variável do tipo da enumeração (enum)

- Uma vez definida a enumeração, uma variável pode ser declarada de modo similar aos tipos já existentes.

```
enum semana s;
```

- e inicializada como qualquer outra variável, usando uma das constantes da enumeração:

```
s = Segunda;
```



ENUMERAÇÕES E CONSTANTES

Enumerações e constantes

- Para o programador uma enumeração pode ser vista como uma lista de constantes, em que cada constante possui um nome significativo.
- Para o compilador cada uma das constantes é representada por um valor inteiro.
 - O valor da primeira constante da enumeração é 0, o da segunda 1, e assim por diante.
- Uma enumeração pode ser usada em qualquer expressão válida com inteiros.

Enumerações e constantes

```
#include <stdio.h>

enum semana {Domingo, Segunda, Terca, Quarta, Quinta, Sexta, Sabado};

int main(void) {
    enum semana s;
    s = Terca;
    printf ("s = %d\n", s);
    return 0;
}
```

- O programa apresentará como saída:

s = 2

Enumerações e constantes

- É possível definir o valor inicial das enumerações atribuindo um valor.
- O programa apresentará como saída:

```
Domingo = 11
Segunda = 12
Terca = 13
Quarta = 14
Quinta = 15
Sexta = 16
Sabado = 17
```

```
#include <stdio.h>
```

```
enum semana {Domingo=11, Segunda, Terca,  
Quarta, Quinta, Sexta, Sabado};
```

```
int main(void) {  
    printf("Domingo = %d\n", Domingo);  
    printf("Segunda = %d\n", Segunda);  
    printf("Terca = %d\n", Terca);  
    printf("Quarta = %d\n", Quarta);  
    printf("Quinta = %d\n", Quinta);  
    printf("Sexta = %d\n", Sexta);  
    printf("Sabado = %d\n", Sabado);  
    return 0;  
}
```

Enumerações e constantes

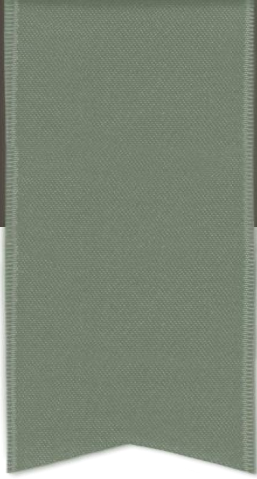
- Outro exemplo.
- O programa apresentará como saída:

```
Domingo = 11
Segunda = 12
Terca = 9
Quarta = 10
Quinta = 11
Sexta = 12
Sabado = 13
```

```
#include <stdio.h>
```

```
enum semana {Domingo=11, Segunda, Terca=9,
Quarta, Quinta, Sexta, Sabado};
```

```
int main(void) {
    printf("Domingo = %d\n", Domingo);
    printf("Segunda = %d\n", Segunda);
    printf("Terca = %d\n", Terca);
    printf("Quarta = %d\n", Quarta);
    printf("Quinta = %d\n", Quinta);
    printf("Sexta = %d\n", Sexta);
    printf("Sabado = %d\n", Sabado);
    return 0;
}
```



COMANDO TYPEDEF

Comando typedef

- A linguagem C permite que o programador defina os seus próprios tipos com base em outros tipos de dados existentes.
- Para isso utiliza-se o comando **typedef**.
- Sua forma geral é:

```
typedef tipo_existente novo_nome;
```

- **tipo_existente** é um tipo básico ou definido pelo programador e **novo_nome** é o nome para o novo tipo que está sendo definido.
- Exemplo:

```
typedef int inteiro;
```

Comando typedef

```
#include <stdio.h>

typedef int inteiro;

int main(void) {
    int i = 5;
    inteiro j;
    j = i * 2;
    printf ("j = %d\n", j);
    return 0;
}
```

Comando typedef

- O comando **typedef** pode ser usado para **simplificar a declaração de um tipo definido pelo programador** (struct, union, etc.).

```
#include <stdio.h>
#include <string.h>

struct pessoa {
    char nome[40];
    char endereco[50];
    float salario;
    int dependentes;
};

typedef struct pessoa Pessoa;

int main(void) {
    Pessoa p;
    strcpy(p.nome, "Pedro Alvares Cabral");
    strcpy(p.endereco, "Rua sobe e desce, nr 0");
    p.salario = 1500.04;
    p.dependentes = 7;
    return 0;
}
```

Comando typedef

- O comando **typedef** pode ser combinado com a declaração de um tipo definido pelo programador em uma única instrução.

```
#include <stdio.h>
#include <string.h>

typedef struct pessoa {
    char nome[40];
    char endereco[50];
    float salario;
    int dependentes;
} Pessoa;

int main(void) {
    Pessoa p;
    strcpy(p.nome, "Pedro Alvares Cabral");
    strcpy(p.endereco, "Rua sobe e desce, nr 0");
    p.salario = 1500.04;
    p.dependentes = 7;
    return 0;
}
```

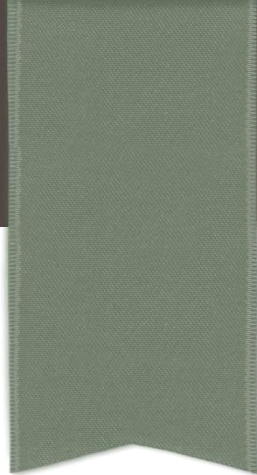
Comando typedef

- Opcionalmente o nome da struct pode ser omitido se estivermos definindo um novo tipo com o comando **typedef**.

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char nome[40];
    char endereco[50];
    float salario;
    int dependentes;
} Pessoa;

int main(void) {
    Pessoa p;
    strcpy(p.nome, "Pedro Alvares Cabral");
    strcpy(p.endereco, "Rua sobe e desce, nr 0");
    p.salario = 1500.04;
    p.dependentes = 7;
    return 0;
}
```



EXERCÍCIOS

Exercícios

1. Crie uma estrutura Ponto para representar as coordenadas de um ponto no plano (posições X e Y). Declare e leia do teclado dois pontos e exiba a distância entre eles.
2. Crie uma estrutura Retângulo. Essa estrutura deverá conter o ponto superior esquerdo e o ponto inferior direito do retângulo. Cada ponto é definido por uma estrutura Ponto, a qual contém as posições X e Y. Faça um programa que declare e leia uma estrutura Retângulo e exiba a área e o perímetro desse retângulo.
3. Crie uma estrutura representando um aluno de uma disciplina. Essa estrutura deve conter o número de matrícula, seu nome e as notas de três provas. Escreva um programa que leia os dados de N alunos e os armazene em um array dessa estrutura (N deverá ser definido com **#define**). Exiba o nome, as notas e a média do aluno que possui a maior média geral. Considere que não haverá empate.
4. Crie uma estrutura representando um atleta. Essa estrutura deve conter o nome do atleta, seu esporte, idade e altura. Escreva um programa que leia os dados de cinco atletas, calcule e exiba os dados do atleta mais alto e do mais novo. Considere que não haverá empate. Os dados dos atletas deverão estar armazenados em um array da estrutura atleta.
5. Crie uma enumeração representando os dias da semana. Agora, escreva um programa que leia um valor inteiro do teclado e exiba o dia da semana correspondente.